Beyond the Visible: Transforming AR Software Engineering Through Empirical Discovery, Metamorphic Testing, and Multi-Agent Intelligence

Dibyendu Brinto Bose

Dissertation submitted to the Faculty of the
Virginia Polytechnic Institute and State University
in partial fulfillment of the requirements for the degree of

Doctor of Philosophy
in
Computer Science and Application

Chris Brown, Chair
Na Meng
Brendan David-John
Denis Gracanin
Xiaoyin Wang

April 24, 2026
Blacksburg, Virginia

Keywords: AR development, Metamorphic testing, Multi Agent System
Copyright 2026, Dibyendu Brinto Bose

Beyond the Visible: Transforming AR Software Engineering Through Empirical Discovery, Metamorphic Testing, and Multi-Agent Intelligence

Dibyendu Brinto Bose

(ABSTRACT)

Augmented Reality (AR) technologies are reshaping interactive computing, yet software engineering support for immersive applications has not kept pace with their growing complexity. Because AR systems produce outputs that depend on dynamic environments, sensor input, and real-world context, the expected outputs are often impossible to determine in advance -a challenge known as the oracle problem -making conventional testing approaches insufficient. To understand how this affects development in practice, we first conduct an empirical study of 48 core AR developers, identifying recurring challenges across the software development lifecycle, with especially persistent difficulties in testing, bug reproduction, and handling environmental variability. Building on these findings, we introduce a metamorphic testing framework for mobile AR that validates context-dependent behavior through domain-specific behavioral relations rather than fixed expected outputs, and we show that the same oracle-avoiding abstraction extends to LLM-generated code, where property-based testing exposes correctness failures that unit tests miss. We then investigate how LLMs themselves can support correctness reasoning in this setting: first, through a stability-driven evaluation framework showing that multi-agent debate improves the reliability of metamorphic relation identification, and then through a repository-aware generation framework that uses hierarchical code context and deliberative refinement to discover new AR-specific relations directly from software repositories. Taken together, this work advances a correctness-oriented

foundation for AR software engineering, grounded in developer practice, expressed through behavioral abstractions, and supported by structured LLM-based reasoning.

Beyond the Visible: Transforming AR Software Engineering Through Empirical Discovery, Metamorphic Testing, and Multi-Agent Intelligence

Dibyendu Brinto Bose

(GENERAL AUDIENCE ABSTRACT)

Augmented Reality (AR) software is becoming more common in areas such as healthcare, education, entertainment, and training. However, building reliable AR applications is difficult because these systems must work correctly while responding to changing physical environments, user movement, sensors, and complex 3D interactions. This makes them much harder to test than traditional software, where the correct output for a given input is usually known in advance. My research studies this problem from several angles. First, I investigated the real-world challenges faced by professional AR developers and found that many of their most persistent difficulties arise during testing and debugging—particularly because the expected behavior of an AR application often cannot be determined in advance. Building on these findings, I developed a testing approach for mobile AR applications that checks whether software continues to behave correctly when inputs or environmental conditions are changed in meaningful ways, rather than requiring a known correct answer. I then explored how modern AI tools, especially large language models (LLMs), can help identify these kinds of behavioral rules more reliably — including studying when AI models are consistent in their reasoning, how multiple AI agents can debate to improve decisions, and how code repository context can help generate better testing knowledge automatically. I also applied this same idea — checking behavior under change rather than checking against a fixed answer — to code written by AI systems, showing that this reveals correctness problems that standard

testing methods often miss. Together, this work contributes new ways to understand, test, and reason about correctness in immersive software, showing how developer-grounded research, advanced testing methods, and AI-based reasoning can be combined to make AR applications more dependable in practice.

Contents

List of Figures xii

List of Tables xiii

1 Introduction 1

1.1 Problem Definition 1

1.2 Research Contributions 3

1.2.1 Empirical Grounding of AR Software Engineering Challenges 4

1.2.2 Behavioral Correctness Modeling for AR Applications and Generated Code 5

1.2.3 LLM-Based Reasoning for Correctness Identification and Discovery . 6

1.2.4 A Correctness-Oriented Perspective on AR Software Engineering . . 7

1.2.5 Contribution to the XR Community 8

1.3 Thesis Statement 10

2 Review of Literature 11

2.1 Challenges Faced by XR Developers 11

2.2 Testing and Validation in AR Applications 12

2.3 Metamorphic Testing and Metamorphic Relation Generation 12

2.4 LLMs for Software Engineering and Correctness Evaluation 14
2.5 Multi-Agent Systems in Software Engineering 15
2.6 Summary of the Literature Gap 17

3 Challenges and Current Practices of Core AR Developers 18

3.1 Chapter Overview 18
3.2 Study Design 20
3.2.1 Survey Design 20
3.2.2 Participants 21
3.2.3 Data Analysis 22
3.3 Results 23
3.3.1 RQ1: What Challenges Do Core AR Developers Face at Different Stages of the SDLC? 23
3.3.2 RQ2: How Do Developers Overcome Challenges Related to AR Development? 28
3.4 Discussion 30
3.4.1 Implications for AR Developer Support 30
3.4.2 Implications for Testing Research 31
3.4.3 The Broader Role of This Chapter in the Dissertation 32
3.5 Threats to Validity 32
3.6 Chapter Summary 33

4 Enhancing Mobile AR Testing through Metamorphic Testing **38**

4.1 Motivation and Problem Statement 38

4.2 Approach 40

4.3 Identified Mobile AR Metamorphic Relations 42

5 Evaluating the Stability of LLMs in Metamorphic Relation Detection **48**

5.1 Chapter Overview 48

5.2 Dataset 51

5.2.1 Data Filtering 51

5.2.2 Dataset Statistics 52

5.3 Methodology 53

5.3.1 Model Selection 53

5.3.2 Generating MR Rephrasings 54

5.3.3 Prompting for Response Generation 55

5.3.4 Assessing Response Consistency 57

5.3.5 Multi-Agent Debate Framework 58

5.4 Results 59

5.4.1 RQ1: How Stable Are LLMs in Identifying Metamorphic Relations Across Rephrasings? 59

5.4.2 RQ2: Can a Multi-Agent Debate Framework Improve Stability? . . . 61

5.4.3 RQ3: To What Extent Does Stability Correlate with Correctness? . . 62

5.5 Discussion . . . 63
5.5.1 Stability Varies by Both Model and Relation . . . 64
5.5.2 Debate Improves Reliability, but Not Equally Everywhere . . . 64
5.5.3 Stability Is Useful, but Only When Interpreted Carefully . . . 65
5.5.4 Consensus Dynamics in Multi-Agent Debate . . . 66
5.6 Threats to Validity . . . 67
5.7 Chapter Summary . . . 68

6 Evaluating Property-Based Testing for LLM-Generated Code 74

6.1 Chapter Overview . . . 74
6.2 Methodology . . . 75
6.2.1 Datasets . . . 75
6.2.2 Prompting for Property Extraction and Code Generation . . . 76
6.2.3 Property-Based Extraction and Mapping . . . 77
6.2.4 Evaluation Strategy . . . 79
6.3 Results . . . 79
6.3.1 RQ1: To what extent can property-guided prompting help LLMs produce executable and behaviorally correct code? . . . 79
6.3.2 RQ2: What differences emerge when LLM-generated code is assessed through conventional benchmark testing versus property-based testing? 81
6.4 Discussion and Implications . . . 82

6.5 Threats to Validity 83

6.6 Chapter Summary 84

7 Repository-Aware Generation of Metamorphic Relations for Augmented Reality Applications 85

7.1 Chapter Overview 85

7.2 Approach 88

7.2.1 Context Configurations for MR Generation 88

7.2.2 Metamorphic Relation Generation Pipeline 90

7.2.3 Reasoning-Orchestrated Debate for MR Refinement 91

7.3 Evaluation Setup 94

7.3.1 Dataset 94

7.3.2 LLMs Configuration 95

7.3.3 Structural Validity and Redundancy Metrics (RQ1) 95

7.3.4 Debate Outcome Analysis (RQ2) 99

7.3.5 Test Oracle Study (RQ3) 99

7.4 Result 101

7.4.1 RQ1: Effect of Context Granularity 101

7.4.2 RQ2: Effect of Multi-Agent Debate 103

7.4.3 RQ3: Generated MR Practicality 104

7.5 Discussion 106

7.5.1 Implications for LLM-Based MR Generation 106
7.5.2 Implications for AR Testing 109
7.5.3 Implications for Practitioners 110
7.6 Chapter Summary 111

8 Future Work **114**

9 Conclusion **118**

Appendices **121**

Bibliography **121**

List of Figures

3.1 Unique challenges within stages of AR application development 35

3.2 Measures developers take when facing challenges 36

3.3 Testing approaches for AR applications 37

5.1 Example of a rephrase of MR definition 55

5.2 Multi-Agent Debate Workflow 70

5.3 Group Stability Score Distribution of the three evaluated models 71

5.4 Group Stability Score Distribution of the multi-agent debate system 72

5.5 Stability scores across metamorphic relations for individual models and the debate framework 73

6.1 Pipeline of property generation and source code generation. 76

7.1 Debate-based MR refinement workflow. Candidate MRs generated under A0, A1, and A2 are aligned by target method, assembled into a debate prompt, evaluated by a critique/judge model, and converted into a final decision: `KEEP_BEST`, `MERGE`, or `DISCARD_ALL`. 113

List of Tables

3.1 AR device and application domain distribution among survey participants . 22

4.1 Identified metamorphic relations for mobile AR testing 44

5.1 Final Dataset Overview 52

5.2 Stability Score Metrics for CodeLlama, StarCoder, and Gemma 59

5.3 Stability Scores per Metamorphic Relation for StarCoder, Gemma, and CodeLlama 60

5.4 Decision Type Distribution Across Multi-Agent Debate. Each decision is made per metamorphic relation, leading to a total of 33,687 $\times$ 7 evaluations. 62

5.5 Spearman's Correlation Between Stability and Correctness 63

5.6 Disagreement Cases Across Metamorphic Relations 67

6.1 Representative correctness-property categories considered in our property-based analysis. 78

6.2 Quality of inferred properties across MBPP and HumanEval (100 sampled problems), reported as percentages of useful, repeated, and omitted constraints. 79

6.3 Outcome of property-based verification on generated programs across MBPP and HumanEval 80

6.4 Distribution of pass@k scores for StarCoder and CodeLlama on the MBPP benchmark 81

6.5 Distribution of pass@k scores for StarCoder and CodeLlama on the HumanEval benchmark 81

7.1 Evaluation Criteria for Test Oracle Study 100

7.2 Effect of Context Granularity on Specificity and Diversity 102

7.3 RQ2: Conflict Identification and Resolution in Multi-Agent Debate ($N = 4496$) 103

7.4 RQ2: Winning Source Distribution After Debate ($N = 3760$) 104

7.5 End-to-End Impact of Debate on MR Quality 104

7.6 Quality of Debate-Refined MRs 105

List of Abbreviations

API Application Programming Interface

AR Augmented Reality & Virtual Reality

AR Augmented Reality

AST Abstract Syntax Tree

CCF Category Choice Framework

CI/CD Continuous Integration / Continuous Deployment

GPT Generative Pre-trained Transformer

GPU Graphics Processing Unit

LLM Large Language Model

MR Metamorphic Relation

MRL Metamorphic Relation Learning

NLP Natural Language Processing

PBT Property-Based Testing

RLHF Reinforcement Learning from Human Feedback

SDLC Software Development Life Cycle

Chapter 1

Introduction

1.1 Problem Definition

Augmented Reality (AR) technologies are becoming increasingly important in domains such as healthcare [105], education [57], entertainment [87], and industrial training [94]. As these systems grow in sophistication and adoption, they place new demands on software engineering practice. AR applications combine software logic with sensor input, spatial reasoning, physical context, and real-time interaction, making their development different from that of more conventional software systems [52, 66, 111]. Despite the growing use of AR, empirical understanding of how these characteristics shape software engineering practice in real-world development settings remains limited, particularly from the perspective of professional developers responsible for building, maintaining, and testing such systems.

Before methods and tools can be tailored to AR, there must be a grounded understanding of what makes these systems difficult to engineer in practice. Prior work has examined topics such as end-user AR creation and interdisciplinary collaboration [10, 38, 62], but it offers limited evidence about the concrete obstacles faced by core AR developers across the software development lifecycle. Our empirical study addresses this gap by showing that core AR developers face challenges across design, implementation, testing, and debugging, especially when they must reason about behavior under dynamic interaction, environmental, and device conditions [17]. These findings suggest that AR software engineering requires support that

accounts for context dependence, interaction complexity, and platform diversity rather than assuming stable and easily reproducible behavior.

This makes testing a natural focus of the thesis. In many AR scenarios, correctness cannot be reduced to a single fixed expected output because system behavior depends on factors such as camera pose, user motion, object placement, occlusion, lighting, and device orientation [10, 17, 62]. Traditional testing approaches often struggle in these settings, since outputs are shaped by changing environmental conditions [103]. As a result, AR software frequently presents an oracle problem, where correctness is difficult to determine through exact output matching alone. In such cases, correctness is often better expressed through behavioral relations that describe how outputs should change when the environment, viewpoint, or interaction conditions are systematically transformed [85, 107]. This creates a need for testing methodologies suited to context-dependent and oracle-deficient software.

At the same time, recent advances in Large Language Models (LLMs) have opened new possibilities for supporting software engineering tasks such as code understanding, test generation, and reasoning about program behavior [29, 59, 113, 129]. Yet their role in AR remains largely underexplored. In immersive systems, useful assistance would require more than generic code generation; it would require reasoning about behavioral transformations, environmental variation, and domain-specific correctness constraints [19, 68]. However, even if LLMs appear capable of recognizing such correctness relations, their usefulness depends on whether they respond consistently across semantically equivalent formulations [106] and whether structured multi-agent reasoning can improve the reliability of their judgments [39, 84]. In correctness-sensitive settings, the value of LLM-based support therefore depends not only on capability, but also on stability, consistency, and the structure of the reasoning process.

These concerns become even more significant when moving from recognizing known correctness relations to generating new ones. Identifying useful metamorphic relations manually is

difficult [73], and recent work suggests that isolated code snippets often provide insufficient context for reasoning about software behavior [79]. Although repository-level context has improved LLM-based understanding of codebases, generating meaningful correctness relations remains challenging [19, 47]. Additional context can still produce overlapping, incomplete, or contradictory candidate relations, making it difficult to derive reliable test oracles [92]. More fundamentally, correctness knowledge in AR is often tied not only to code structure, but also to interaction patterns and environmental conditions, which are difficult to capture through either traditional testing alone or naive LLM prompting.

Taken together, this thesis is motivated by a unified gap in current software engineering support for immersive systems: the field lacks an empirically grounded understanding of how AR development challenges manifest in practice, principled ways to express and validate correctness when fixed test oracles are unavailable, and reliable AI-based mechanisms for discovering and refining such correctness knowledge at scale. This thesis addresses that gap by connecting empirical developer insight, metamorphic and property-based reasoning, and structured LLM-based workflows to advance more reliable, scalable, and context-aware software engineering practices for AR systems.

1.2 Research Contributions

This thesis advances the software engineering of augmented and virtual Reality (AR) systems by developing a correctness-oriented perspective grounded in three interconnected lines of contribution: an empirical characterization of where AR testing fails in practice, the formalization of behavioral correctness relations for immersive and AI-generated software, and a systematic investigation of how large language models can support correctness reasoning at scale. Each contribution is supported by peer-reviewed publications and builds directly

on the preceding one.

1.2.1 Empirical Grounding of AR Software Engineering Challenges

This thesis begins by establishing an empirical foundation that motivates and scopes the problem space [17]. Through a survey of 48 core AR developers—professionals whose primary expertise lies in building and maintaining immersive applications—we systematically identify challenges that recur across the full software development lifecycle rather than at any single stage. Using open coding with an inter-rater agreement of Cohen's $\kappa = 0.83$ (indicating near-perfect agreement), we categorize these challenges into three SDLC phases. Design-phase difficulties include immersive experience creation and cross-device scalability. Implementation challenges include handling high-dimensional sensor data and ensuring cross-platform compatibility in 3D interaction design. Testing challenges—the most directly relevant to this thesis—include the inability to determine expected outputs in advance, difficulty reproducing environment-dependent bugs, and the effect of environmental variability on test outcomes.

This contribution is foundational in two ways. First, it provides direct empirical evidence that AR testing is structurally different from conventional software testing: it is characterized by dynamic environments, sensor-driven behavior, and the absence of reliable oracles. Second, it motivates the specific correctness abstractions developed in subsequent contributions, grounding theoretical choices in documented practitioner need.

1.2.2 Behavioral Correctness Modeling for AR Applications and Generated Code

A central contribution of this thesis is the development of correctness-oriented testing frameworks that operate without fixed expected outputs—directly addressing the oracle problem identified in the empirical study.

The first framework targets mobile AR applications [18]. Rather than attempting to specify ground-truth outputs for environment-dependent behavior, we formalize a set of domain-specific *metamorphic relations* (MRs) that capture how AR application outputs should change under defined input transformations. Seven relations are identified and implemented: *Object Scaling with Distance*, *Visibility and Occlusion*, *Overlapping with Game Objects*, *Orientation Consistency Relative to Gravity*, *Varying Rotation*, *Raycast within Boundary*, and *Check Correct Object Instantiation*. These relations encode spatial consistency properties of AR pipelines—such as proportional scaling and occlusion adjustment—that can be validated through behavioral transformation rather than output matching, making testing feasible even when exact rendering outcomes cannot be predetermined.

The second framework extends correctness modeling to a different and increasingly critical domain: code generated by large language models [15]. Through evaluation of StarCoder and CodeLlama on the HumanEval and MBPP benchmarks, we demonstrate that unit-test-based evaluation systematically overstates the correctness of generated code, because it verifies only specific input-output pairs without capturing deeper structural constraints. Applying property-based testing (PBT) as an alternative evaluation strategy, we show that between 18–23% of solutions that pass unit tests fail fundamental correctness properties, and that 30–32% only partially satisfy them. This contribution establishes that the same oracle-avoiding insight underlying metamorphic testing—validating behavioral properties across

transformed inputs rather than verifying exact outputs—extends naturally to the evaluation of AI-generated code, where ground truth is similarly elusive.

Together, these two contributions establish behavioral relations and properties as a unifying correctness abstraction that applies across both immersive AR applications and AI-generated software artifacts.

1.2.3 LLM-Based Reasoning for Correctness Identification and Discovery

Having established the value of metamorphic relations as a correctness abstraction, this thesis then investigates how LLMs can support the process of identifying, evaluating, and generating such relations at scale. This line of work proceeds in two stages.

The first stage concerns *MR identification*: assessing whether LLMs can recognize when an established metamorphic relation applies to a given code snippet [16]. We introduce a stability-driven evaluation framework that measures whether LLM judgments remain consistent across semantically equivalent rephrasings of MR definitions—an important proxy for genuine semantic understanding in the absence of ground truth. Evaluating StarCoder, CodeLlama, and Gemma across 33,687 verified AR source files, we find that code-focused models achieve meaningfully higher stability than general-purpose ones (CodeLlama: 0.7988, StarCoder: 0.6684, Gemma: 0.3309), but that stability alone does not guarantee correctness: CodeLlama exhibits high consistency while achieving only 52.3% accuracy against human-labeled ground truth. We then demonstrate that a multi-agent debate mechanism—where models iteratively challenge and refine each other's MR classifications—improves both stability (average: 0.8309) and correctness (81.2%), and achieves the strongest alignment with human judgment (Spearman's $\rho = 0.73$). This contribution establishes that reliability in

correctness reasoning requires not just individual model capability but structured deliberation.

The second stage advances from MR *identification* to MR *discovery*: automatically generating new metamorphic relations directly from AR software repositories, where no ground-truth relations are assumed to exist. We propose a repository-aware MR generation framework that organizes contextual information hierarchically across repository, class, and method levels. Across 142 Unity-based AR repositories and 5,167 class–method targets, hierarchical context (A2) produces the broadest coverage and lowest redundancy, with an 86.7% unique MR rate, while demonstrating that context *organization* matters as much as context *volume*: unstructured flat retrieval (A1) degrades source traceability below even the method-only baseline despite having access to more raw information. To reconcile the 14,916 candidate MRs produced across context configurations—which exhibit contradictions in 79.0% of cases and redundancies in 51.2%—we introduce a reasoning-orchestrated deliberation mechanism that reduces the exact duplicate rate from 13.3% to 1.3% and selects or synthesizes context-aware relations in 88.2% of outcomes. A manual oracle study of 141 debate-refined MRs confirms that the resulting relations are practically usable, achieving high validity (1.80/2) and testability (1.82/2) scores, with 61.7% rated high-quality across all dimensions. Together, these two stages show how LLMs can progress from recognizing known correctness relations to scalably discovering and refining new ones.

1.2.4 A Correctness-Oriented Perspective on AR Software Engineering

Taken together, the contributions of this thesis constitute a correctness-oriented perspective on AR software engineering—one that addresses the oracle problem not by avoiding it but by

systematically reframing what correctness means in domains where exact expected outputs cannot be known in advance.

The empirical study establishes that AR testing is structurally oracle-deficient, and characterizes where this deficiency has the greatest practical impact. The metamorphic and property-based contributions provide principled ways to express correctness under this deficiency—through behavioral relations for AR applications and through general properties for AI-generated code—establishing a shared abstraction that spans two distinct and increasingly important software domains. The LLM-based contributions demonstrate that this abstraction can be operationalized at scale: correctness relations can be reliably identified in existing code through stability-aware evaluation and multi-agent deliberation, and they can be discovered from scratch in large software repositories through structured context-aware generation and reasoning-based refinement.

1.2.5 Contribution to the XR Community

This dissertation contributes to the XR community by providing a testing abstraction that matches the direction of emerging XR platforms. Recent developments such as Google's Android XR show that future XR applications will increasingly rely on AI-assisted headsets and glasses, spatial understanding, hand and gaze interaction, and real-world anchoring. These systems create new testing challenges because correctness is no longer defined only by a fixed input and output. Instead, behavior depends on user motion, physical surroundings, device pose, interaction modality, and how virtual content remains aligned with the real world.

The metamorphic relations developed in this dissertation provide a practical way to test such behavior. Rather than requiring developers to know the exact expected output in advance,

each MR defines how the system should behave when the input, viewpoint, environment, or interaction state changes. For example, an XR application may not have one fixed visual output for object placement, occlusion, or hand-based interaction. However, it should still satisfy relational expectations: a placed object should remain within a valid spatial boundary, occlusion should update consistently as the viewpoint changes, object scale should vary predictably with distance, and interaction outcomes should remain stable under controlled changes in hand pose, gaze direction, or device movement.

This input-output relational structure is especially useful for building reusable XR test knowledge. An MR database could organize common correctness relations for spatial anchoring, raycasting, object placement, occlusion, hand interaction, gaze interaction, gravity alignment, and distance-based scaling. Developers building applications for mobile AR, XR headsets, or smart glasses could then reuse these relations instead of designing new test oracles from scratch for every application. In this sense, the MR database becomes a practical bridge between XR interaction challenges and executable testing support.

Another important advantage is transferability. Because each MR is expressed as an input transformation and an expected output relation, it can be translated into source-code-level tests across different XR devices and platforms. The same high-level relation can be implemented using phone-based AR APIs, headset pose data, hand-tracking information, gaze events, or spatial anchors depending on the target system. This separates the correctness property from device-specific implementation details and makes the approach adaptable to both current AR applications and upcoming XR devices.

Overall, this work contributes to the XR community by showing how interaction-heavy and environment-sensitive behaviors can be captured as reusable, testable relations. As XR systems move toward AI-assisted, spatially aware, and wearable experiences, metamorphic testing offers a scalable way to validate correctness when fixed expected outputs are difficult

or impossible to define.

In combination, these contributions move AR software engineering toward a more systematic and scalable treatment of correctness validation: one grounded in practitioner evidence, expressed through behavioral abstractions that sidestep the oracle problem, and increasingly supported by AI-assisted reasoning workflows.

1.3 Thesis Statement

Because AR applications exhibit context-dependent, environment-sensitive behavior that—as empirically demonstrated through developer practice—renders conventional oracle-based testing insufficient, ensuring their correctness requires formalizing behavioral relations and properties that express correctness without relying on fixed expected outputs, and applying LLM-based structured reasoning to identify, evaluate, and scalably discover such relations directly from real-world software repositories.

Chapter 2

Review of Literature

2.1 Challenges Faced by XR Developers

Prior research has examined AR development from several complementary perspectives, but most of this work has focused on end users, tool support, or interdisciplinary collaboration rather than on the day-to-day software engineering concerns of core developers. For example, Krauß et al. [62] studied collaborative extended reality (XR) development and highlighted coordination challenges across interdisciplinary roles. Nebeling et al. [38] and Gandy et al. [42] emphasized limitations in AR authoring tools and the need for better support for non-expert creators. Similarly, Ashtari et al. [10] examined the barriers faced by end-user developers building AR applications, including limited programming knowledge, tool complexity, and weak support for iterative creation. Other work, such as Speicher et al. [112], identified technical issues in cross-device AR settings, including field-of-view constraints, gesture-recognition difficulties, and other platform-specific limitations.

Taken together, these studies show that immersive application development is difficult, but they primarily emphasize end-user creation, collaboration, or tooling constraints rather than the software engineering practices of developers whose main responsibility is to implement, maintain, test, and debug AR systems. More broadly, recent surveys and domain reviews reinforce the view that AR development poses distinct engineering challenges because such systems must coordinate physical context, device capabilities, and interactive behavior. How-

ever, before our work, there was limited empirical evidence on how *core* AR developers themselves experience these challenges across the software development lifecycle. This gap matters because difficulties in design, implementation, and testing directly shape what kinds of methods and tools are needed for immersive software engineering.

2.2 Testing and Validation in AR Applications

Prior work on AR testing has explored several forms of validation support, including model-based testing, oracle prediction, generative exploration, debugging tools, and usability-oriented feedback systems [26, 42, 62, 112]. Representative examples include VR-ReST for functional testing [30], ARCHIE for automated interface-testing logs [66]. Although these approaches strengthen particular aspects of testing, they do not fully resolve the broader oracle problem that arises when correct behavior cannot be specified through fixed expected outputs alone [35]. Consequently, much of the literature remains focused on scenario-specific test design [95] or narrower forms of debugging and interaction testing [101]. A more general correctness-oriented perspective, one that captures expected AR behavior through transformations and behavioral consistency, remains underdeveloped. This gap directly motivates the use of metamorphic testing in this thesis.

2.3 Metamorphic Testing and Metamorphic Relation Generation

Metamorphic Testing (MT) is a well-established approach for addressing the oracle problem, where the correct output of a program is difficult or infeasible to determine directly [107].

Instead of relying on fixed expected outputs, MT uses *metamorphic relations* (MRs), which specify necessary relationships among multiple executions of a program under controlled input transformations. Foundational work introduced MRs as a practical way to validate software behavior when conventional test oracles are unavailable, and subsequent studies explored a variety of strategies for generating effective source test cases, including random testing, manual-plus-random combinations, genetic algorithms for path coverage, and equivalence-class partitioning [23, 60, 119].

Over time, MT has been applied across a wide range of domains, including numerical software, image processing, computer graphics, and machine learning [58, 88]. For example, prior work has used MT to test image-processing programs [85], mesh simplification software [20], and morphological image operations [54]. These studies demonstrate that MT is especially valuable in domains where correctness is easier to express as a relation between executions than as a single expected output.

At the same time, the literature shows that identifying useful MRs remains a major challenge. Constructing meaningful relations often requires substantial domain knowledge [68], understanding of implementation details, and careful reasoning about how input transformations should affect outputs [11]. Prior surveys and MR-generation research describe search-based [126], pattern-based [108], mutation-based [109], and learning-based approaches [114], but they also emphasize that MR construction remains labor-intensive and only partially automated in practice. This difficulty becomes more pronounced in large software repositories, where relevant behavioral properties may be distributed across multiple methods, classes, and interacting components [121].

Within the context of AR, this challenge is especially important. As established later in this dissertation, AR behaviors such as boundary consistency, scaling, occlusion, and orientation can be formulated as metamorphic relations, making MT a natural fit for immersive soft-

ware testing [18]. However, once such domain-relevant relations are recognized, a broader challenge remains: how to discover new MRs systematically rather than rely on manually defined ones. Existing work has largely focused on defining or detecting known relations, rather than generating new MRs from broader repository-level context [92]. This gap motivates the repository-aware and LLM-supported MR reasoning developed in the later chapters of this thesis.

2.4 LLMs for Software Engineering and Correctness Evaluation

Large language models are increasingly used in software engineering for tasks such as code generation [78], documentation [44], requirements engineering [113], test generation [129], and program reasoning [59]. Recent studies show that these models can support many development workflows, but they also expose persistent weaknesses, including hallucination, shallow reasoning, and limited trustworthiness in generated outputs [106]. In code generation research, evaluation still relies heavily on benchmark-driven unit testing, such as pass@k on datasets like HumanEval and MBPP [41, 74]. Although useful, this style of evaluation emphasizes predefined input-output examples and may overestimate correctness when deeper logical or structural constraints are not exercised [91].

To address these limitations, recent work has begun to explore richer ways of evaluating LLM outputs. Mixed-initiative frameworks such as EvalGen help users define evaluation criteria while incorporating human feedback [110]. Other work has examined whether LLMs can substitute for human annotators in software engineering studies, showing that model agreement can sometimes approach human inter-rater agreement [7]. Researchers have also

proposed trust-oriented evaluation frameworks such as TrustScore, which measures behavioral consistency between model outputs and the model's implicit knowledge [127], as well as benchmark suites such as TruthEval that probe reliability on carefully constructed statements with known truth values [61]. Together, these studies reflect a broader shift away from simple accuracy-based evaluation toward reliability-, consistency-, and trust-aware assessment.

This shift is especially important in correctness-oriented software engineering tasks, where a plausible answer is not sufficient unless the reasoning behind it is dependable. In oracle-deficient settings such as AR, this problem becomes even more severe because correctness is often difficult to establish through fixed expected outputs alone. Our work builds on this emerging direction by treating *stability across semantically equivalent rephrasings* as an evaluation lens for LLM-based reasoning about metamorphic relations. In parallel, our property-based testing work addresses a related gap in code generation evaluation by showing that solutions passing conventional unit tests may still violate important behavioral properties. Together, these contributions position this thesis within a broader effort to evaluate LLMs not only by whether they produce correct-looking answers, but by whether their reasoning remains consistent and behaviorally grounded under changing formulations.

2.5 Multi-Agent Systems in Software Engineering

Recent work has shown that multi-agent LLM architectures can improve software engineering tasks by distributing reasoning across specialized roles. For example, MAGIS [115] uses role-specific agents such as manager, developer, and QA engineer to resolve GitHub issues at the repository level, outperforming standalone models on SWE-bench [56]. MetaGPT [48] similarly organizes agents through standardized operating procedures, using intermediate

verification to reduce cascading hallucinations and improve solution coherence. Other studies have applied multi-agent frameworks to software engineering activities such as qualitative data analysis [99] and Agile-inspired development workflows [82], suggesting that role specialization and structured interaction can improve reasoning quality across a range of tasks.

At the same time, a growing body of work argues that many software engineering problems require repository-level reasoning rather than isolated function-level analysis. Recent approaches use hierarchical summaries [79], graph-based code representations [19], and structured contextual retrieval [47] to help LLMs reason over large codebases despite context-window limitations. These studies show that project structure, class relationships, and inter-method dependencies are important for tasks such as code search [92], bug localization [79], and specification mining [47]. This is especially relevant for AR applications, where behavior often emerges from interactions among multiple components rather than from a single method in isolation [130].

However, existing work leaves two important gaps. First, prior research on metamorphic relations has largely focused on detecting predefined relations from isolated code snippets, rather than generating new relations from repository-level program context. Second, although debate-based and multi-agent systems have been used to refine existing artifacts, they generally assume that a candidate solution already exists. In contrast, MR discovery requires generating, comparing, and reconciling multiple candidate behavioral relations without access to ground truth. This gap motivates the repository-aware and reasoning-orchestrated approach developed in this thesis.

2.6 Summary of the Literature Gap

Taken together, prior work establishes that AR software presents distinctive development and testing challenges, that metamorphic testing is a promising way to address oracle-deficient validation, that LLMs can support software reasoning but require stronger evaluation, and that repository-aware and multi-agent methods improve reasoning over large codebases. Yet these threads have remained largely disconnected. Existing AR studies do not sufficiently characterize the software engineering concerns of core developers; AR testing research does not provide scalable mechanisms for identifying correctness relations; LLM-based correctness work has emphasized isolated tasks or predefined relations; and repository-aware reasoning has not been fully integrated with oracle construction for immersive systems. This thesis bridges those gaps by combining empirical developer insight, metamorphic and property-based correctness abstractions, stability-driven LLM evaluation, and repository-aware deliberative reasoning into a unified correctness-oriented perspective on AR software engineering.

Our study extends the use of MAS to evaluate LLMs' ability to detect MRs in AR-related source code. While prior research has leveraged MAS for development automation and collaborative problem-solving, our focus is on whether structured debate among LLM agents improves stability in MR identification.

Chapter 3

Challenges and Current Practices of Core AR Developers

3.1 Chapter Overview

As AR technologies continue to expand beyond entertainment and gaming into domains such as education, healthcare, industry, and military training, the software engineering demands placed on immersive systems have become increasingly complex. AR applications must integrate real-time sensing, spatial interaction, environmental awareness, multimodal feedback, and device-specific rendering constraints in ways that differ substantially from conventional desktop or mobile software. Despite this growth, the software engineering knowledge base for AR remains relatively underdeveloped, particularly with respect to the practical challenges faced by the developers who build and maintain such systems.

This chapter addresses that gap through an empirical study of *core AR developers*. In this dissertation, the term *core AR developers* refers to professionals whose primary work and expertise center on the design, implementation, and maintenance of AR applications. This distinguishes them from occasional or peripheral contributors who may interact with immersive technologies only as part of a broader role. Focusing on this population is important because their experiences provide direct insight into the obstacles that shape real-world immersive software development and, in turn, reveal where current software engineering

methods fall short.

The chapter serves a foundational role in the dissertation. The broader argument of this dissertation is that AR software engineering requires methods that are grounded in the actual realities of immersive development rather than borrowed uncritically from traditional software domains. Before proposing specialized testing frameworks or LLM-based reasoning methods, it is necessary to understand what practitioners themselves perceive as the major pain points in the AR software development lifecycle. This chapter therefore establishes the empirical basis for the later contributions of the dissertation. In particular, the challenges identified here directly motivate the metamorphic testing framework presented in the following chapter and the later exploration of LLM-supported reasoning for AR-related software analysis.

To investigate the experiences of core AR developers, I conducted a survey study with 48 practitioners and analyzed their responses using qualitative open coding. The study is guided by the following research questions:

RQ1: What challenges do core AR developers face at different stages of the software development lifecycle?

RQ2: How do developers overcome challenges related to AR development?

The results show that the most prominent challenges arise in three major phases of the software development lifecycle: *design, implementation, and testing.* Developers report difficulties related to immersive experience design, design scalability, high-dimensional sensor and scene data, cross-platform compatibility, complex 3D interaction logic, uncertainty about expected outcomes, difficulty reproducing bugs, and environmental variability during testing. The chapter also shows that developers cope with these problems primarily through

community knowledge sources, documentation, simulators, and device-based trial-and-error testing.

The remainder of the chapter is organized as follows. Section 3.2 describes the study design, including survey construction, participant recruitment, and qualitative analysis. Section 3.3 presents the findings for both research questions. Section 3.4 interprets these findings in relation to broader software engineering challenges in immersive systems. Section 3.5 discusses threats to validity. Finally, Section 3.6 summarizes the chapter and explains how the findings motivate the next stage of the dissertation.

3.2 Study Design

This study was designed to examine how experienced AR developers describe the practical challenges they face during real-world development. Because the goal was to surface both recurring patterns and practitioner-specific details, I adopted a survey-based design with a substantial open-ended component. This allowed respondents not only to identify problem areas but also to describe them in their own words, making it possible to capture challenges that may not be visible through predefined checklists alone.

3.2.1 Survey Design

The survey was organized around major stages of the software development lifecycle (SDLC), specifically *design, implementation, and testing* [97]. These phases were selected because they capture the parts of development most likely to be affected by the distinctive characteristics of AR systems, including immersive interaction design, platform dependency, environmental sensitivity, and real-time feedback.

The questionnaire began with background questions about participant demographics, professional role, AR development experience, and the types of devices and application domains in which they worked. These questions were necessary for ensuring that the sample reflected active practitioners with current experience in immersive technology development.

The survey then moved into a set of open-ended questions designed to elicit detailed reflections on developer challenges and coping strategies. In total, ten survey questions were open-ended. These questions focused on the kinds of problems developers encounter during design, implementation, and testing, as well as the resources and practices they use when those problems arise. Additional questions asked about the AR platforms they used and the contexts in which they developed applications. Before deployment, the study instrument was reviewed and approved by the Institutional Review Board (IRB) of my institution.

3.2.2 Participants

Participants were recruited through online AR developer communities on Facebook,[1] LinkedIn,[2] and Discord.[3] These platforms were chosen because they host active practitioner communities and provide access to developers working across diverse AR ecosystems.

The final sample consisted of 48 core AR developers. Among them, 35 participants (73%) identified as male and 13 (27%) identified as female. Participants reported between four months and seven years of AR development experience, with an average of 2 years and a median of 2.2 years. All respondents were actively working in AR at the time of the study. Their roles included Associate Extended Reality (XR) Developer, Senior XR Developer, Junior XR Developer, Team Lead, AR Researcher, and XR Researcher. Participants came

[1] https://www.facebook.com/
[2] https://www.linkedin.com/
[3] https://discord.com/

from organizations such as AIVerse, Deloitte, Sigmoid AI, Bosch Rexroth, Megagon Lab, and the authors' primary institution.

The sample also reflected diversity in both application focus and development environment. Twenty-three participants reported working only on AR applications, eighteen worked only on VR applications, and seven worked on both AR and VR. Table 3.1 summarizes the devices participants used and the domains in which they developed applications.

Table 3.1: AR device and application domain distribution among survey participants

Device	**Responses**	**Domain**	**Responses**
HTC Vive	15	Entertainment	19
HoloLens	11	Business-related	13
Magic Leap 2	9	Medical-oriented	11
Varjo XR-3	9	Educational	8
Mobile apps	5	Research prototype	5
Oculus Quest	3	Health	2
Samsung Gear VR	2	Financial	1
		User study	1

Note: Mobile apps refer to AR apps for smartphones. Participants could report multiple devices and domains.

The table shows that participants worked across a wide variety of device ecosystems and application areas. This breadth is important because it suggests that the study captures challenges that are not limited to a single platform or usage context, but instead reflect broader development realities in immersive computing.

3.2.3 Data Analysis

To analyze the qualitative survey responses, I used open coding [31], a method that has been shown to be effective in empirical software engineering research for surfacing themes from practitioner data [5, 6]. The aim was to identify both the major categories of challenges reported by developers and the strategies they used to address those challenges.

Two experienced raters with backgrounds in AR and software engineering independently coded the responses. Their analysis followed three steps:

- Each rater first read the responses carefully and highlighted keywords or phrases that captured important ideas.
- Based on these keywords, each rater assigned an initial theme to the relevant statement.
- The raters then grouped related themes into broader categories corresponding to development challenges and coping practices.

After the independent coding phase, disagreements were resolved through discussion until a final categorization was reached. To assess the consistency of the initial coding process, Cohen's Kappa coefficient [28] was computed prior to reconciliation.

3.3 Results

This section presents the study findings for the two research questions. I first discuss the major challenges reported by core AR developers across different phases of the SDLC, and then describe the strategies developers use to cope with those challenges.

3.3.1 RQ1: What Challenges Do Core AR Developers Face at Different Stages of the SDLC?

The first research question focuses on identifying the distinctive challenges that arise during AR development. The open coding process achieved a Cohen's Kappa score of 0.83, which indicates *almost perfect agreement* according to Landis and Koch [64]. Figure 3.1 summarizes

the major challenge categories identified across the design, implementation, and testing phases.

A notable pattern across the findings is that developers did not describe AR development as difficult in a single generic sense. Instead, they emphasized that different kinds of challenges emerge at different stages of development. This reinforces the idea that immersive software engineering is not merely conventional software engineering plus 3D interfaces, but a domain in which each lifecycle stage is shaped by distinct technical and experiential constraints.

Design Challenges

Two design-related themes were especially prominent: *immersive experience design* and *scalability of design*.

Immersive Experience Design. Developers frequently described the difficulty of designing experiences that feel genuinely immersive rather than merely functional. Unlike traditional applications, AR systems must account for presence, realism, user perception, embodied interaction, and multisensory engagement. One participant explained:

> "Making an experience that feels real and engaging for users is tough. It's not just about visuals but about making it feel immersive across all senses" [P15].

This challenge reflects the fact that AR design extends beyond interface layout or feature composition. Developers must think in terms of spatial and experiential coherence, which makes design more subjective, more difficult to evaluate early, and more dependent on user perception than in conventional software projects.

Scalability of Design. A second challenge concerned the difficulty of ensuring that a design works well across multiple devices, platforms, and hardware capabilities. Participants described how differences in display properties, tracking capabilities, field of view, and interaction methods complicate design reuse and consistency. As one participant noted:

> "It's really hard to make sure a design looks and works great on every device. Different devices mean different challenges" [P22].

This finding suggests that scalability in AR design is not simply a matter of responsive layout or cross-platform rendering. Instead, it involves preserving experiential quality under widely varying hardware and environmental conditions.

Implementation Challenges

Within implementation, three recurring themes emerged: *handling high-dimensional data*, *cross-platform compatibility*, and *3D interaction design complexity*.

Handling High-Dimensional Data. AR applications often consume and process large amounts of heterogeneous input, including sensor streams, depth information, spatial mappings, pose data, and environmental context. Developers described this as a major source of implementation difficulty, particularly when real-time performance must be maintained. One participant stated:

> "Dealing with all the data from sensors and depth maps without slowing things down is a big challenge" [P9].

This challenge highlights the data-intensive nature of immersive systems. Developers are

not simply implementing logic over abstract program states; they are managing continuously evolving real-world signals that must be processed efficiently and correctly.

Cross-Platform Compatibility. Participants also emphasized the burden of making AR applications work consistently across different hardware and operating systems. This problem extends beyond standard portability concerns because each AR platform may introduce its own conventions, tracking limitations, API behaviors, and rendering expectations. One participant explained:

> "Making sure the app works well on different devices and platforms is a real struggle. Each one has its own quirks" [P30].

This theme indicates that implementation work in AR is often deeply entangled with platform-specific assumptions, which makes portability costly and error-prone.

3D Interaction Design Complexity. A third implementation challenge involved designing and implementing interactions that feel intuitive in three-dimensional space. Participants contrasted this with conventional 2D interaction design, noting that spatial interaction introduces additional complexity in gesture interpretation, object manipulation, movement, and depth perception. One respondent observed:

> "Getting 3D interactions to feel natural and intuitive is really hard. It's not like working with 2D where things are simpler" [P18].

This challenge reflects a broader mismatch between traditional interaction paradigms and immersive environments. Developers must translate user intent into spatial behavior in ways that remain understandable, responsive, and physically plausible.

Testing Challenges

The testing phase surfaced three especially significant challenges: *knowing the output beforehand*, *bug reproduction*, and *environmental variability*.

Knowing the Output Beforehand. A recurring challenge in testing AR systems is the difficulty of determining what the correct outcome should be before execution. Because application behavior depends on physical context, user behavior, and environmental conditions, expected outputs are often not fixed in the same way they are for conventional programs. One participant described this problem as follows:

> "It's often hard to anticipate what the app will do in different scenarios. For example, when testing a VR game, it's tough to predict how different actions will affect the virtual environment" [P44].

This finding is particularly important for the broader dissertation, because it directly points to the oracle problem in immersive systems and motivates the later use of metamorphic testing.

Bug Reproduction. Developers also reported that bugs in AR systems can be extremely difficult to reproduce. Since failures may depend on specific sequences of user actions, device states, or environmental triggers, developers often struggle to recreate the exact conditions under which a problem occurs. As one participant explained:

> "Reproducing a bug is really tricky. For instance, a certain glitch only happens when users interact with specific objects in a particular sequence, which is hard to recreate consistently during testing" [P12].

This challenge reduces the effectiveness of conventional debugging and regression workflows, which often assume that failures can be recreated deterministically.

Environmental Variability. Finally, developers noted that AR systems often behave differently under different physical conditions. Lighting, room layout, tracking surfaces, movement patterns, and sensor noise can all affect system behavior. One participant remarked:

> "I once had a problem where a feature worked fine in the lab but failed in a different setting with natural sunlight" [P27].

This theme highlights one of the defining properties of immersive computing: software behavior cannot be fully separated from the environment in which it runs. As a result, testing requires reasoning not only about code but also about context.

3.3.2 RQ2: How Do Developers Overcome Challenges Related to AR Development?

The second research question examines how developers respond to the challenges identified above. Rather than relying primarily on formalized AR-specific engineering processes, participants reported depending heavily on informal knowledge sources and tool-supported experimentation.

Figure 3.2 shows that developers most commonly rely on three resources: question-and-answer websites, developer forums, and device-specific documentation. Platforms such as Stack Overflow and Stack Exchange provide community-generated solutions to implementation problems, while device-specific forums, such as Magic Leap's developer forum, help

developers navigate hardware-specific issues. Official documentation for tools such as Unity and WebXR also serves as an important reference source.

The overlap analysis further shows that these resources are not used uniformly. Fifteen participants (31%) reported relying only on Q/A websites, twelve (25%) relied only on documentation, and eight (17%) relied only on developer forums. This suggests that developers often assemble their own ad hoc support workflows rather than following a standardized problem-solving process. In other words, when difficulties arise, developers compensate for missing methodological support by piecing together guidance from external communities and vendor ecosystems.

Current Measures for Testing AR Applications

Developers were also asked specifically about how they test AR applications in practice. As shown in Figure 3.3, the three most common approaches were using device simulators, relying on existing IDE-based testing features, and deploying software directly to physical devices for manual testing.

Nineteen participants reported using simulators tailored to specific devices. Fifteen reported using testing support already available within their integrated development environment. Fourteen reported deploying software to the actual target device and testing it there. These responses suggest that AR testing remains highly practice-driven and tool-constrained. Developers rely on what their current ecosystem makes available, but none of these measures fully resolves the core testing difficulties identified in RQ1, especially uncertainty about expected outputs, environmental sensitivity, and reproducibility.

3.4 Discussion

The findings of this chapter reveal that AR development challenges are not isolated technical annoyances, but systematic problems that cut across the software development lifecycle. They also show that current coping strategies are largely reactive. Developers rely on documentation, community support, simulators, and direct device testing because formalized engineering support for immersive systems remains limited. This section discusses what these findings imply for both research and practice.

3.4.1 Implications for AR Developer Support

The design and implementation findings suggest that current development support for AR remains insufficiently aligned with the realities of immersive systems. Developers are expected to create engaging multisensory experiences, reason about complex spatial interactions, process high-dimensional environmental data, and maintain compatibility across fragmented hardware ecosystems. These challenges go well beyond the assumptions built into many conventional software engineering tools and workflows.

Some of these findings resonate with prior work on cross-platform application development [21], but the present study shows that the problem is amplified in immersive systems because platform differences affect not only functionality but also user experience, sensor interpretation, and interaction design. This implies a need for more specialized tooling, such as debugging environments that can capture and replay spatial context, support for analyzing sensor-rich execution traces, and frameworks that help developers reason about 3D interaction behaviors across devices.

The findings also suggest that developer support should not focus solely on code production.

Design support is equally important. Since immersive experience design is difficult to evaluate using conventional software artifacts alone, future tools may need to better integrate prototyping, simulation, behavior capture, and user interaction replay to help developers iterate on experience quality as well as correctness.

3.4.2 Implications for Testing Research

The testing results are especially significant because they expose structural limitations in how AR systems are currently validated. Developers struggle to know the correct output beforehand, to reproduce failures reliably, and to account for environmental variability. Taken together, these challenges indicate that many AR testing problems are fundamentally *oracle-limited* and *context-dependent*.

This has important implications for testing research. Existing approaches such as VR-ReST [111], usability-feedback-driven evaluation [40], and log-based testing tools such as *ARCHIE* [66] address parts of the problem, but they do not fully resolve the difficulty of validating behavior in unpredictable real-world conditions. Similarly, prior work from end-user perspectives [10] has pointed to the need for more interactive testing support, but the present findings show that professional developers also face deep uncertainty during testing.

One promising direction is the use of methods that do not depend on exact expected outputs. For example, metamorphic testing [24] offers a way to reason about correctness through relations between transformed inputs and outputs rather than through fixed oracle values. This directly addresses the challenge reported by developers who cannot know the output beforehand but can still reason about how behavior should change under certain transformations. In this sense, the present empirical findings do more than describe developer pain points: they provide concrete justification for the specialized testing framework introduced in the

next chapter.

3.4.3 The Broader Role of This Chapter in the Dissertation

Within the overall dissertation, this chapter plays a foundational role. It establishes that the problems addressed later in the dissertation are not hypothetical or artificially constructed. Instead, they are grounded in the lived experience of practitioners working on AR systems. The later chapters on metamorphic testing and LLM-supported reasoning can therefore be understood as responses to real and recurring development constraints rather than as disconnected technical proposals.

More specifically, three findings from this chapter directly shape the rest of the dissertation. First, the difficulty of knowing expected outputs motivates the use of oracle-free testing approaches. Second, the difficulty of bug reproduction and environmental variation motivates a shift toward relation-based and context-aware validation strategies. Third, the heavy reliance on ad hoc external resources suggests that developers need stronger automated support, which later chapters explore through LLM-based reasoning and multi-agent systems.

3.5 Threats to Validity

Several threats to validity should be considered when interpreting the results of this study.

Internal Validity. The study relies on self-reported survey responses, which may be affected by recall bias or by participants emphasizing particularly memorable challenges over routine ones. To mitigate this, the survey included open-ended questions that encouraged respondents to describe concrete experiences rather than simply select from predetermined

categories.

Construct Validity. The study focuses on challenges organized around the SDLC phases of design, implementation, and testing. Although this framing is useful and aligns with the dissertation's broader focus, it may not capture every possible dimension of AR development work, such as long-term maintenance, deployment, or organizational coordination. Nonetheless, the selected phases cover the development activities most directly relevant to the dissertation's later testing and validation contributions.

External Validity. The sample includes 48 developers recruited from online communities and therefore may not represent the full population of AR developers worldwide. In particular, developers working in highly specialized industrial settings or within closed commercial ecosystems may face additional challenges not captured here. At the same time, the diversity of roles, platforms, and domains in the sample improves confidence that the identified themes are not restricted to a single niche of immersive development.

Conclusion Validity. The categorization of qualitative responses depends on the coding process. Although open coding introduces some interpretive subjectivity, this threat is mitigated by the use of two independent raters and the high inter-rater agreement score of 0.83.

3.6 Chapter Summary

This chapter presented an empirical study of 48 core AR developers to understand the challenges they face across the software development lifecycle and the strategies they use to cope

with those challenges. The findings show that immersive systems introduce distinctive difficulties during design, implementation, and testing. Developers struggle to create scalable immersive experiences, manage high-dimensional data, maintain cross-platform compatibility, implement intuitive 3D interactions, predict expected outcomes, reproduce bugs, and test reliably across variable environments. In response, they rely heavily on documentation, developer communities, simulators, and device-based experimentation.

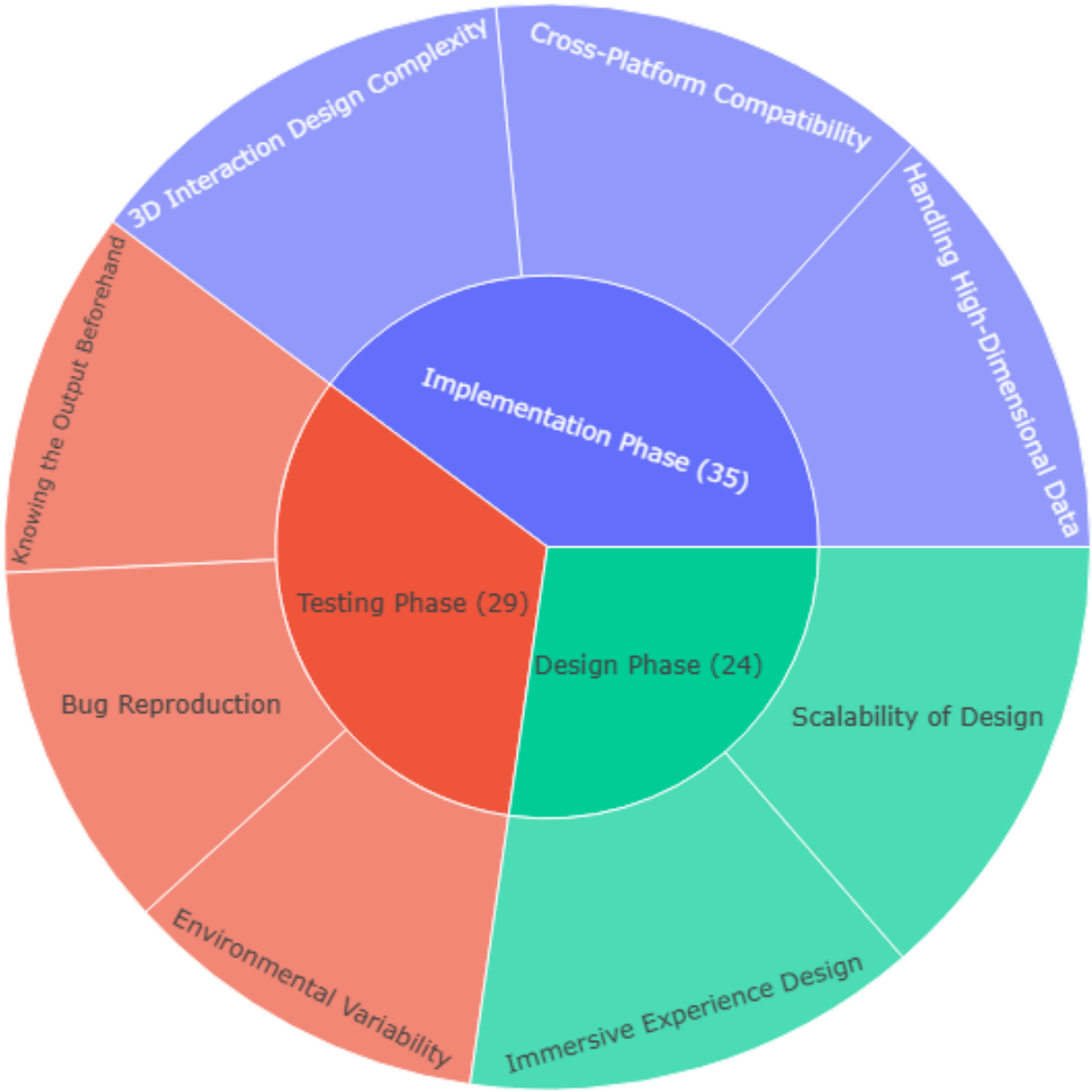


Figure 3.1: Unique challenges within stages of AR application development

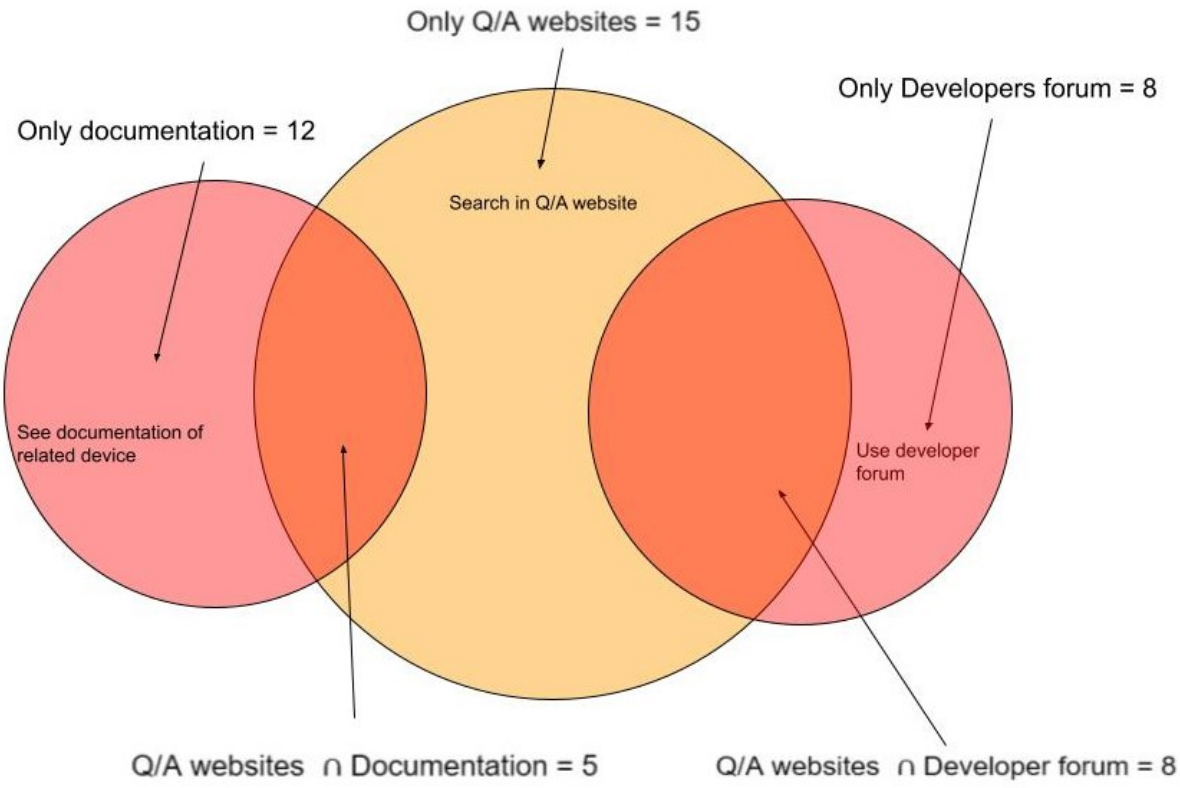


Figure 3.2: Measures developers take when facing challenges

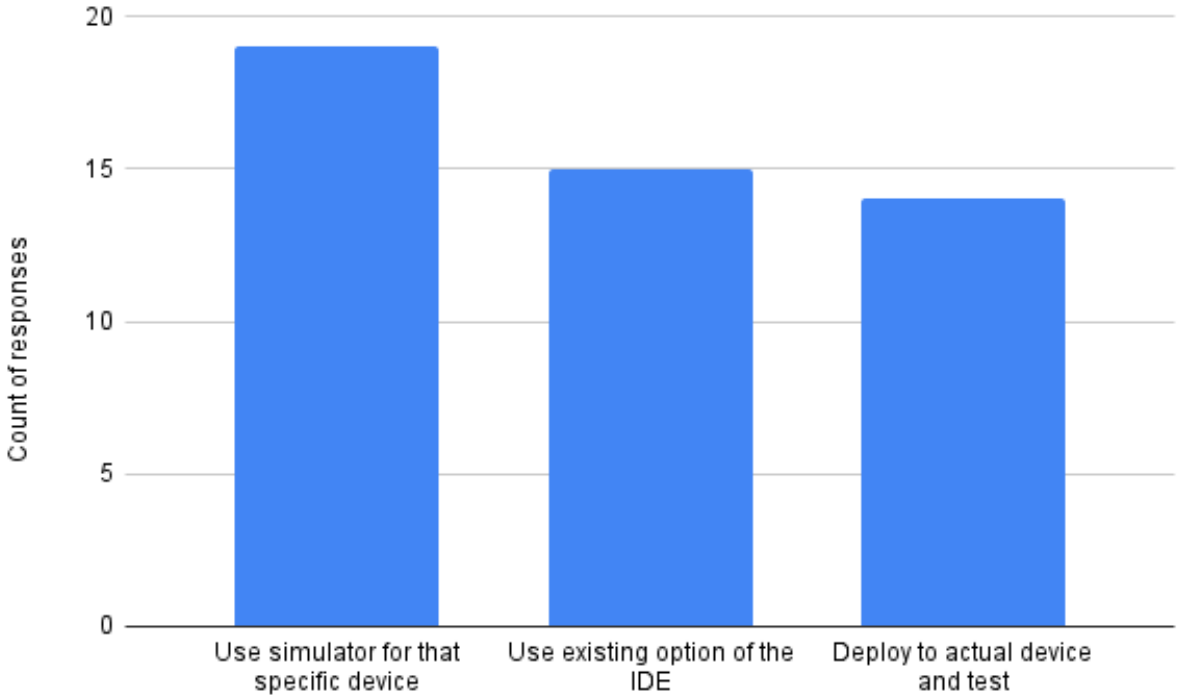


Figure 3.3: Testing approaches for AR applications

Chapter 4

Enhancing Mobile AR Testing through Metamorphic Testing

4.1 Motivation and Problem Statement

AR technologies have become increasingly influential across a wide range of domains, including gaming [3], entertainment [4], education [57, 86], the military [94], and medicine [63, 105]. By combining physical surroundings with digital content, these technologies create immersive and interactive experiences for users. Their role is not limited to consumer applications; AR systems are also used in settings where correctness and dependability are essential. In medicine, for instance, AR supports tasks such as preoperative planning and intraoperative guidance, where reliability directly affects patient safety [90]. In educational settings, AR can improve learning by delivering interactive and immersive materials, which must operate consistently to ensure meaningful instructional value [9]. Likewise, military training environments rely on AR-based simulations that must behave reliably to support realistic and effective training exercises [33, 65]. The expanding use of AR across these domains highlights the growing need for effective and dependable testing strategies.

Despite their benefits, AR applications remain difficult to validate. Testing these systems is particularly challenging because they involve dynamic 3D interactions, environmental dependency, and real-time system responses. These characteristics limit the usefulness of

many traditional software testing techniques. Prior research points to the need for stronger testing practices to preserve software quality and expected system behavior in AR systems [52, 66, 111]. Even so, important obstacles remain. Existing work reports that end-users often lack sufficient development knowledge to effectively validate these systems [10], while current tools are still insufficient for addressing the specialized demands of VR testing [103].

To examine these issues more closely, our preliminary work included a broad survey of AR developers. The survey revealed several recurring challenges in testing mixed-reality applications, such as difficulty interpreting and visualizing test outcomes, handling feature-rich and complex application requirements, and coping with the decreasing usefulness of traditional Q&A forums. Developers and end-users also described the difficulty of working with unpredictable behaviors and complicated input-output relationships that frequently arise in AR environments.

To address these challenges, we propose *Metamorphic Testing* (MT), a testing technique that does not require a conventional test oracle [24]. Instead of checking correctness against a single known expected output, MT relies on Metamorphic Relations (MRs), which specify how outputs should change in response to controlled changes in input. This makes MT particularly suitable for AR systems, where the diversity of interactions and environmental variability often make traditional test validation difficult.

As an initial step, our work concentrates on mobile AR applications. We selected mobile AR as the starting point for two main reasons. First, our earlier survey indicated that mobile devices are among the most commonly used platforms for AR development. Second, mobile AR systems were readily accessible for our experimentation. Our objective is to improve the testing process by incorporating MT principles in a way that better reflects the realities of mobile AR development and use. By identifying and applying MRs designed specifically for mobile AR scenarios, we seek to establish a testing framework that can address the concrete

challenges encountered by both developers and end-users.

Through this work, we position Metamorphic Testing as a practical and robust solution for AR application testing. Our goal is to address the distinctive difficulties of testing AR applications and, in doing so, support the broader advancement of dependable AR technologies. As adoption continues to accelerate and application contexts continue to diversify, there is a growing demand for testing approaches that can match this complexity. Our proposed methodology is intended to offer a scalable and realistic way to improve the reliability and overall quality of AR experiences.

4.2 Approach

Metamorphic Testing (MT) offers a distinct perspective on software testing by focusing on relationships between program inputs and outputs rather than relying solely on explicitly known expected results [24]. This technique is particularly useful for addressing two persistent problems in software testing: generating additional test cases from existing ones and handling the Oracle problem, where determining the correct output for a given input is difficult or infeasible [25]. At the center of MT are Metamorphic Relations (MRs), which describe how output behavior should change when input conditions are systematically modified. These relations provide a structured basis for testing by allowing testers to assess whether the observed behavior remains consistent with expected relational properties.

A simple example helps clarify this idea. Consider a search engine that returns a set of results for the query “car”. A reasonable MR would state that the query “electric car” should produce a subset of those results, since electric cars represent a narrower category within cars. This example demonstrates why MT is valuable in settings where exact expected outputs are difficult to define in advance. Rather than depending on direct output verification, MT

derives follow-up test cases from existing source test cases and checks whether the observed outputs satisfy the relevant MR [124]. Applying this idea to AR systems requires careful identification of relations that reflect the behavior of immersive and interactive environments. In this work, we adopt an ad hoc MR identification strategy informed by prior research [25]. Although more systematic approaches, such as swarm optimization, have been explored for MR discovery [126], much of the existing MT literature still relies on less formal methods. Even so, prior studies have shown that ad hoc approaches can still be effective in uncovering faults [76]. Building on that foundation, we adapt these ideas for the specific needs of AR testing.

The MRs we propose for AR testing are designed around the distinctive characteristics of mobile AR applications. Each relation defines relevant inputs, expected outcomes, and transformations suited to AR interaction scenarios. For example, one MR checks whether virtual objects preserve appropriate relative positioning as users move through an AR scene. Another evaluates whether object scaling behaves consistently when the user's viewpoint or distance changes. To identify such relations, we conducted a detailed examination of common AR interactions together with the practical expectations of developers and end-users. Our focus was placed on core AR behaviors, including object placement, scaling, and occlusion. Based on these behaviors, we constructed a testing framework intended not only to expose faults but also to support a more stable and predictable user experience across varying conditions.

The process of identifying and refining MRs is both iterative and collaborative. We continuously revise the proposed relations using feedback from AR developers as well as results obtained through empirical testing. This repeated refinement helps ensure that the selected MRs remain useful as AR platforms and development practices evolve. By integrating perspectives from both developers and end-users, our approach stays grounded in real-world

usage and practical testing concerns, making it more adaptable to the changing needs of AR applications.

Overall, our use of Metamorphic Testing aims to strengthen the state of testing for AR applications. MRs provide a systematic mechanism for detecting faults in complex interactive environments and for improving confidence in system behavior. The main contribution of this work is the development of an enhanced testing framework tailored to the demands of AR systems, with the broader goal of enabling more reliable and user-centered AR applications.

4.3 Identified Mobile AR Metamorphic Relations

Our process for identifying metamorphic relations (MRs) was shaped by findings from both end-users [10] and developers. Prior empirical studies involving end-users [10], together with insights from developers, revealed several recurring challenges that arise from the immersive and interactive nature of augmented reality (AR) applications. These challenges include determining whether an object is placed inside a detected plane, ensuring that newly spawned objects do not conflict with existing ones, verifying that objects are instantiated correctly, checking whether environment tracking functions properly, and confirming that application behavior remains stable across different surroundings. In practice, addressing such issues often depends heavily on repeated experimentation and trial-and-error.

In response to these challenges, our work introduces a set of carefully defined metamorphic relations intended to directly address the issues reported by practitioners. Using this feedback as a foundation, we identified a set of core AR application behaviors that could serve as the basis for MR design. The formulation of these MRs was further informed by two of Nielsen's usability heuristics [1]: *Recognition rather than recall* and *Error prevention*. The first heuristic influenced our design by encouraging test scenarios and expected outcomes

that are easy to interpret, thereby reducing the amount of implementation-specific knowledge developers must remember during testing. For example, MRs such as "Object Scaling with Distance" and "Raycast within Boundary" express behaviors that are intuitive and directly observable. The second heuristic shaped our design by emphasizing the prevention of common AR failures before they affect user experience. MRs such as "Overlapping with Game Objects" and "Visibility and Occlusion" are intended to proactively capture problems related to object interference and visual inconsistency, thereby improving system dependability.

As one example, the MR "*Object Scaling with Distance*" reflects both of these heuristics. It specifies that when the distance between an object and the AR camera changes, the apparent scale of that object should also adjust in a way that preserves realistic visual behavior. This supports recognition rather than recall because developers can readily anticipate the expected outcome without memorizing exact numeric scaling factors or distance thresholds. At the same time, it contributes to error prevention by explicitly describing how scaling should behave under distance changes. For instance, if a small movement of the camera causes an object to shrink dramatically, such behavior would be recognized as inconsistent and therefore indicative of a fault. Table 4.1 presents the MRs identified for AR applications. We further describe each metamorphic relation in detail below.

- **Raycast within Boundary**: This metamorphic relation verifies that an object spawned on an initial AR plane continues to remain inside the valid boundary of newly detected planes as the camera moves. Because the AR camera may identify multiple planes while scanning the physical space, the object location must be re-evaluated whenever a new plane is detected. To support this, the system recalculates the object's position by computing the dot product for each plane transform and checking whether the result falls within an acceptable tolerance range, repeating this process across detected

Table 4.1: Identified metamorphic relations for mobile AR testing

MR Name	**Original Behavior**	**Input Transformation**	**Expected Transformed Behavior**
Raycast within Boundary	A spawned object should remain inside the detected plane boundary.	Move the AR camera so that additional planes are detected.	The placed object should remain consistently positioned within the boundary of the newly detected plane.
Overlapping with Game Object	A touch event should spawn a game object on the detected surface.	Attempt to spawn another object where an object already exists.	The new object should not be spawned if it overlaps with an existing object.
Visibility and Occlusion	A virtual object placed partly behind another object should be partially occluded.	Move the camera to view the object from different angles.	Occlusion should update consistently as the viewing perspective changes.
Object Scaling with Distance	A spawned object should appear at a size proportional to its distance from the camera.	Move the camera closer to or farther from the object.	The object should scale up or down according to the camera-object distance.
Orientation Consistency Relative to Gravity	An object should maintain its intended orientation relative to gravity.	Rotate or tilt the AR device.	The object should adjust its orientation to remain consistent with gravity.
Varying Rotation	A simulated object rotation should produce the expected rotation angle.	Change the initial angle or rotation speed.	The resulting rotation angle should remain within an acceptable threshold.
Check Correct Object Instantiation	A touch action should instantiate the intended object.	Not applicable.	The instantiated object should match the object selected by the touch action.

planes. This MR aligns with *Recognition rather than recall* because it gives developers a clear and interpretable way to verify whether an object remains within plane limits. It also supports *Error prevention* by reducing the risk of objects appearing outside intended spatial boundaries.

- **Overlapping with Game Object**: This relation is intended to prevent one spawned game object from overlapping with another. When a user touch event triggers the spawning of a new object, the system must first determine whether another object already occupies that location. If an existing game object is found, the new object should not be created. This behavior is implemented in Unity through a combination of layer masking and raycasting, which allows the system to detect previously spawned objects rather than only interacting with the plane layer. This MR reflects *Recognition rather than recall* by making the rule against overlap simple and observable, helping developers quickly understand expected interaction behavior. It also contributes to *Error prevention* by blocking object creation in occupied areas, thereby avoiding both visual clutter and functional interference.

- **Visibility and Occlusion**: This MR addresses the expected occlusion behavior of virtual content when it interacts with surrounding objects. If a virtual object is positioned partially behind a real-world object, the hidden portion should be appropriately occluded. As the camera viewpoint changes, the occlusion effect should also update correctly so that the virtual object remains visually consistent from different perspectives. This behavior relies on depth sensing and spatial mapping to interpret the physical environment and manage occlusion in a realistic way. This MR supports *Recognition rather than recall* by ensuring that visibility changes follow predictable rules that developers can easily inspect. It also advances *Error prevention* by dynamically updating occlusion relationships and thereby reducing unrealistic rendering artifacts.

- **Object Scaling with Distance**: This relation checks whether the apparent size of a virtual object changes proportionally with its distance from the camera. When the camera moves closer to the object or farther away from it, the object should scale in a corresponding manner. This is implemented by measuring the distance between the camera and the object and then modifying the object's scale accordingly. This MR supports *Recognition rather than recall* because the relationship between size and distance is intuitive and easy for developers to verify. It also promotes *Error prevention* by establishing explicit expectations for scaling behavior, helping prevent visually incorrect object resizing.

- **Orientation Consistency Relative to Gravity**: This MR ensures that a virtual object maintains an orientation that remains consistent with gravity. When the AR device is tilted or rotated, the object should update its orientation in a way that preserves alignment with gravitational direction. To achieve this, the system continuously monitors device orientation and adjusts the object accordingly. This relation supports *Recognition rather than recall* because gravity-aligned object behavior is natural and straightforward to interpret. It also contributes to *Error prevention* by correcting orientation continuously, which helps avoid unrealistic cases in which objects appear tilted incorrectly or suspended unnaturally.

- **Varying Rotation**: This relation focuses on preserving consistent rotation behavior. When rotation is simulated, the resulting angle should remain stable even when the initial rotation angle or rotation speed changes. The resulting angle is expected to stay within a defined threshold, and relevant rotation parameters are checked to verify this consistency. This MR supports *Recognition rather than recall* by defining understandable criteria for validating rotation. It also supports *Error prevention* by enforcing acceptable thresholds and monitoring rotation-related values, thereby reducing the

likelihood of incorrect or unstable rotational behavior.

- **Check Correct Object Instantiation**: This MR verifies that the object created in response to a touch event corresponds correctly to that touch input. The validation is performed through a direct comparison between the interaction and the instantiated object, ensuring that the expected object is produced. This relation supports *Recognition rather than recall* by making the creation process easy to observe and validate. It also advances *Error prevention* by checking that touch inputs result in the correct object, thereby reducing the possibility of unintended or incorrect instantiation.

Chapter 5

Evaluating the Stability of LLMs in Metamorphic Relation Detection

5.1 Chapter Overview

The previous chapter established a set of MRs for mobile AR applications as a way to test systems whose expected outputs are difficult to specify in advance. Having defined these AR-specific behavioral relations, the next question is whether LLMs can reliably reason about them when analyzing source code. This chapter addresses that question by studying the *stability* of LLM-based MR detection.

Recent advances in artificial intelligence have significantly influenced software engineering practice. Large language models are now used for code generation [29, 129], debugging [59, 113], code improvement suggestions [125], and software maintenance [75]. They have also shown promise in testing-related tasks, including automated test generation [93], bug detection [77], and development workflows that approximate human-level reasoning in constrained settings [84]. This growing adoption is reflected in recent developer surveys showing that a large majority of practitioners are already using, or planning to use, AI tools in development and testing workflows [2]. Despite this momentum, the reliability of LLMs in software testing remains uncertain, especially when tasks require consistent reasoning about program behavior rather than surface-level code completion.

This concern is especially important in domains where testing is already difficult. AR applications provide one such setting. Their behavior depends not only on source code but also on physical context, device motion, sensor input, and environmental variation [26, 62]. As discussed in the previous chapter, these properties make AR software difficult to validate using conventional input-output testing alone [10, 17, 38]. Metamorphic testing offers a suitable alternative because it evaluates whether required behavioral relations hold across transformed conditions rather than requiring exact expected outputs [24, 107]. Prior work has already shown how such relations can be formulated for mobile AR systems [18]. What remains unclear, however, is whether LLMs can recognize those relations consistently when the same MR is described in different ways.

This chapter addresses that gap by proposing a stability-driven evaluation framework for LLM-based MR detection. Rather than focusing solely on accuracy, the chapter examines whether models produce consistent judgments across semantically equivalent rephrasings of the same MR. This is important because a model that truly understands the behavioral logic of an MR should not change its judgment simply because the wording changes. If its responses vary substantially across rephrasings, then the model may be responding to linguistic cues rather than reasoning about the underlying transformation itself.

The central premise of this chapter is that stability can serve as an informative signal of robustness in LLM reasoning for software engineering tasks. In domains such as AR, where ground truth is difficult to establish at scale, stability provides an additional lens through which model behavior can be evaluated. This chapter therefore studies not only how stable different code-capable LLMs are in MR detection, but also whether structured multi-agent debate can improve that stability and whether higher stability aligns with correctness.

The chapter is guided by the following research questions:

- **RQ1:** How stable are LLMs in identifying metamorphic relations across different rephrasings?
- **RQ2:** Can a multi-agent debate framework improve the stability of metamorphic relation identification in LLMs?
- **RQ3:** To what extent does stability in metamorphic relation detection correlate with correctness?

To answer these questions, this chapter evaluates three open-source LLMs—StarCoder, CodeLlama, and Gemma—on a large corpus of mobile AR source files. It measures response consistency across rephrased MR definitions, then introduces a multi-agent debate mechanism in which models critique one another's MR judgments before a final decision is made. The results show that models differ substantially in stability, that debate improves overall consistency, and that stability alone is not always a reliable proxy for correctness. Together, these findings deepen the dissertation's broader argument that LLMs can assist AR software engineering, but only when their reasoning behavior is evaluated and constrained carefully.

The main contributions of this chapter are as follows:

- A stability-oriented evaluation framework for assessing whether code-focused LLMs can identify established AR metamorphic relations consistently.
- An empirical comparison of three LLMs—StarCoder, CodeLlama, and Gemma—under MR rephrasing variations.
- A multi-agent debate framework for refining MR judgments and improving response stability.

- An analysis of the relationship between stability and correctness in LLM-based MR detection.

5.2 Dataset

To study LLM behavior in a realistic AR setting, I constructed a dataset of open-source mobile AR projects collected from GitHub the GitHub API. The goal of the dataset construction process was not merely to gather a large volume of code, but to build a benchmark that reflects actively maintained and technically viable mobile AR software. Repositories were therefore selected using both domain-specific and quality-oriented filtering criteria.

The search targeted repositories associated with mobile AR development through terms such as "AR Foundation," "ARCore," "ARKit," and "Augmented Reality." Because mobile AR ecosystems span multiple platforms, the dataset includes projects written in C#, Java, Kotlin, and Swift. To focus the study on repositories that are more likely to reflect real development practice rather than abandoned prototypes, only projects with at least 10 GitHub stars and commits pushed after January 2023 were retained. Data collection continued until February 2025. In addition, repositories were restricted to those released under widely used open-source licenses, including MIT[1], Apache-2.0[2], and BSD-3-Clause[3].

5.2.1 Data Filtering

The initial collection process yielded 97,510 source files. Since repository metadata alone does not guarantee usability, an additional validation phase was performed to filter out

[1] https://opensource.org/license/mit
[2] https://www.apache.org/licenses/LICENSE-2.0
[3] https://opensource.org/license/bsd-3-clause

projects that were unlikely to support meaningful code analysis. This step focused on both compilability and project activity.

For build verification, Unity-based projects were checked using *Unity Test Runner*[4], Android projects were examined using *Gradle Build Scanner*[5], and iOS projects were validated using *Xcodebuild*[6]. Repositories that failed to compile were excluded. In addition, projects with minimal commit activity or a very limited development history maintained by a single contributor were removed to reduce noise from inactive or trivial repositories. After this filtering process, the final dataset consisted of 33,687 verified and compilable source code files.

5.2.2 Dataset Statistics

Table 5.1 summarizes the distribution of verified files by programming language.

Table 5.1: Final Dataset Overview

Language	**Verified Files**
C#	12,681
Java	7,322
Kotlin	9,448
Swift	4,236
Total	33,687

Beyond file counts, the dataset contains 253,630 import statements, 12,920,490 external dependency references, and 2,048,121 lines of comments. The average file length is 5,601 characters. These characteristics make the dataset sufficiently diverse for studying LLM reasoning in mobile AR code while ensuring that the underlying projects are recent, compilable, and representative of practical development.

[4] https://docs.unity3d.com/2017.4/Documentation/Manual/testing-editortestsrunner.html
[5] https://docs.gradle.org/current/userguide/build_scans.html
[6] https://developer.apple.com/documentation/xcode/build-system

5.3 Methodology

This chapter evaluates whether LLMs can detect AR-specific metamorphic relations in source code in a way that is stable across rephrasings. The methodology begins from the AR MR definitions established in the previous chapter and uses them as the target concepts for model evaluation. Because definitive ground truth is difficult to establish for large-scale MR applicability judgments, the analysis emphasizes consistency of reasoning. The central assumption is that if a model genuinely understands the logic behind a given MR, then semantically equivalent formulations of that MR should lead to similar judgments.

5.3.1 Model Selection

Three open-source models were selected for the study: StarCoder-7B[7], CodeLlama-7B[8], and Gemma-7B[9]. These models were chosen to represent different training emphases and to allow comparison between code-specialized and more general-purpose language models.

StarCoder is trained on a broad range of programming languages and is optimized for code generation, making it a suitable baseline for source-code-oriented reasoning. CodeLlama, derived from Llama-2 and adapted for code-centric tasks, was selected because it is expected to show stronger performance on structured code understanding. Gemma was included to examine whether a general-purpose model can generalize to MR reasoning despite not being primarily optimized for code analysis. Together, these three models provide a useful spectrum for studying how model specialization relates to stability in MR detection.

[7] https://huggingface.co/blog/starcoder
[8] https://github.com/meta-llama/codellama
[9] https://huggingface.co/google/gemma-7b

5.3.2 Generating MR Rephrasings

A key part of the methodology is to test each model against multiple formulations of the same MR. Prior work suggests that rephrasing can influence LLM behavior [122], yet manually crafting many alternative MR statements is difficult. This is particularly true because MR construction itself already demands considerable effort and domain knowledge [37, 43, 67], and complex programs may involve large numbers of possible relations [51].

To create controlled variations, I first generated three rephrasings per MR: the original statement and two alternative formulations. Rephrasing was performed using `FLAN-T5-XXL`[10], chosen for its strong instruction-following ability and effectiveness in controlled text generation [27]. Because rephrasings must remain semantically faithful while still introducing linguistic variation, cosine similarity filtering was applied using the `all-MiniLM-L6-v2` embedding model[11] [118]. Similarity thresholds between 0.4 and 0.9 were used to exclude variants that either drifted too far from the original meaning or remained near-duplicates [100]. Figure 5.1 illustrates an example of this rephrasing process.

After generating and filtering the initial rephrasings, I analyzed the variance in model responses across them. The observed variance was 0.0860 for CodeLlama, 0.0401 for Gemma, 0.0631 for StarCoder, and 0.1548 for the multi-agent debate system. A power analysis based on these variances was then used to estimate how many additional rephrasings would be needed for statistical robustness. The resulting sample size estimates were 15 for CodeLlama, 7 for Gemma, 11 for StarCoder, and 27 for the multi-agent debate condition. Because the largest model group required the greatest coverage, the final experiment used 4,812 rephrasings.

[10] https://huggingface.co/google/flan-t5-xxl
[11] https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

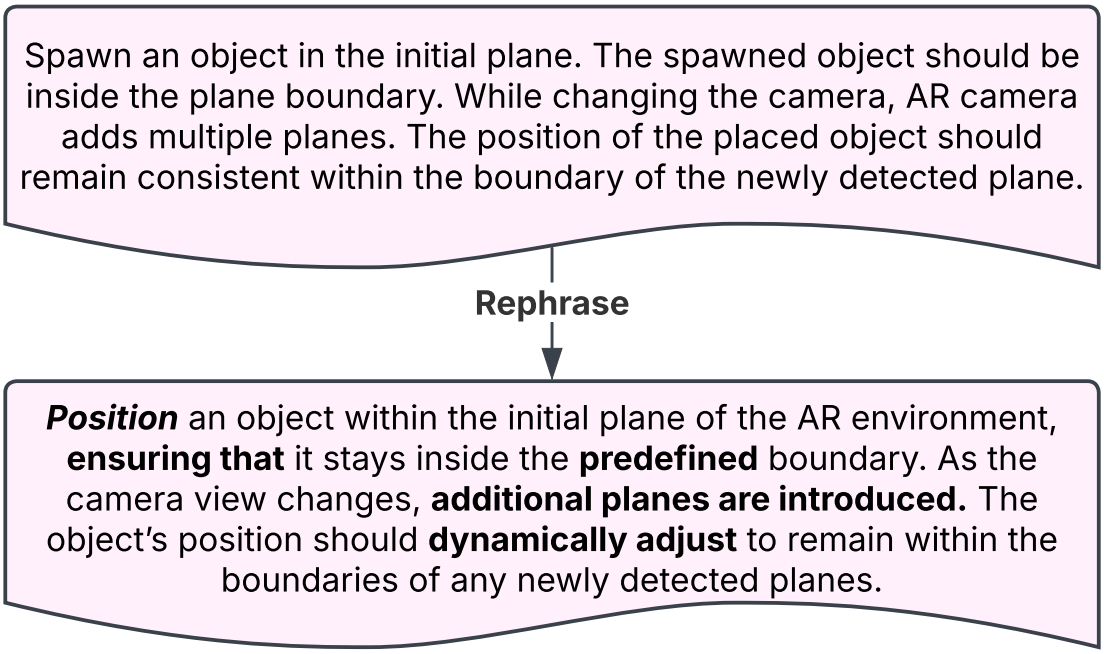


Figure 5.1: Example of a rephrase of MR definition

5.3.3 Prompting for Response Generation

To elicit structured MR judgments, the study used instruction-based prompting [96]. Each prompt provided a cleaned mobile AR code snippet together with the name of a target MR, then asked the model whether the MR was applicable to that code. The objective was not merely to obtain a yes-or-no label, but to force the model to make an interpretable decision grounded in the code.

Before prompting, the source code was cleaned to reduce irrelevant noise. This included removing comments [89], redundant whitespace [50], and formatting inconsistencies [34]. The intent of this preprocessing was to help the model focus on meaningful structural cues

rather than superficial formatting artifacts.

The prompt consisted of three parts:

- **Task Definition:** The model is explicitly told that it is analyzing a mobile AR code snippet for a specific metamorphic relation.
- **Code Context:** The cleaned code is presented as the basis for judgment.
- **Guided Response Format:** The model is instructed to provide (1) a Yes/No answer, (2) a brief explanation, and (3) the key code elements supporting the judgment.

MR Classification Prompt:

Prompt Template

You are analyzing a mobile AR code snippet for applicability of the Metamorphic Relation (MR): {MR Name}.

Given Cleaned Code:

{cleaned_code}

Question:

Does this code have scope for applying the MR '{MR Name}'?

Answer Format:

- Yes or No
- Brief explanation
- Key elements from the code that support your answer

An illustrative example is shown below for the MR *Raycast within Boundary*. In this case, the expected response identifies raycasting as behavior that can be constrained by boundary-aware reasoning.

Example Input and Expected Output:

MR Name: "Raycast within Boundary"

Cleaned Code:

```
public class Example {
    void Update() {
        Ray ray = Camera.main.ScreenPointToRay(Input.mousePosition);
        if (Physics.Raycast(ray, out hit)) {
            Debug.Log("Object Hit");
        }
    }
}
```

Expected Model Output:

```
Yes, this code has scope for applying the MR `Raycast within Boundary'
because it involves raycasting, which should be confined within predefined
boundaries to avoid unintended behaviors.
```

5.3.4 Assessing Response Consistency

The primary evaluation target in this chapter is response stability. To quantify stability, I measured how similar a model's responses were across different rephrasings of the same MR for the same code sample. Rather than comparing all responses to a single reference answer, the approach measures consistency internally within each group of rephrased prompts.

For a sample group g, the group-level consistency score is defined as:

$$S_g = \frac{1}{M} \sum_{i=1}^{M} \sum_{j=i+1}^{M} \text{sim}(R_{g_i}, R_{g_j}) \tag{5.1}$$

where M is the number of MR variations in the group and $\text{sim}(R_{g_i}, R_{g_j})$ denotes cosine similarity between the two responses. The overall stability across all groups is then computed as:

$$S_{\text{final}} = \frac{1}{G} \sum_{g=1}^{G} S_g \tag{5.2}$$

where G is the total number of groups. This formulation allows the evaluation to focus on whether a model reasons consistently across semantically equivalent prompts rather than whether it matches a single gold-standard answer in every instance.

5.3.5 Multi-Agent Debate Framework

In addition to evaluating models independently, this chapter investigates whether structured deliberation can improve consistency. To do so, I implemented a multi-agent debate framework using StarCoder and CodeLlama, the two models that showed stronger stability than Gemma in the single-model setting.

The debate framework is illustrated in Figure 5.2. It follows four stages:

1. **Initial Assessment:** StarCoder and CodeLlama independently judge whether a given MR applies to the code.

2. **Convergence Check:** If both models agree, the process ends immediately.

3. **Challenge Phase:** If they disagree, each model critiques the other's reasoning and refines its position.

4. **Final Judgment:** A decision agent selects the final outcome using the weighted responses informed by the stability results from RQ1.

The debate continues until the models converge or a predefined maximum of three rounds is reached. This three-round limit was selected based on prior work showing that multi-agent debate often reaches stable conclusions within a small number of iterations and that longer debates may add computational cost without proportional reasoning benefit [36, 45, 120].

5.4 Results

This section presents the results for each research question in turn.

5.4.1 RQ1: How Stable Are LLMs in Identifying Metamorphic Relations Across Rephrasings?

Table 5.2 reports the overall stability scores for the three models. CodeLlama achieved the highest mean stability score (0.7988), followed by StarCoder (0.6684), while Gemma performed substantially worse (0.3309). These results indicate that the ability to produce consistent MR judgments varies considerably across models.

Table 5.2: Stability Score Metrics for CodeLlama, StarCoder, and Gemma

Model	Mean Stability Score	Standard Deviation	Interquartile Range (IQR)
CodeLlama	0.7988	0.1199	0.1752
StarCoder	0.6684	0.1446	0.1941
Gemma	0.3309	0.0984	0.1106

A finer-grained view is shown in Table 5.3, which breaks stability down by metamorphic relation. CodeLlama remains consistently ahead across all seven MRs, although the specific

gap varies by relation. For example, it performs especially well on *Raycast within Boundary*, whereas all models are relatively weaker on *Overlapping with Game Object*. These differences suggest that some MRs are inherently easier for LLMs to reason about than others, likely due to the clarity or frequency of the corresponding spatial patterns in code.

Table 5.3: Stability Scores per Metamorphic Relation for StarCoder, Gemma, and CodeLlama

Metamorphic Relation	**StarCoder**	**Gemma**	**CodeLlama**
Raycast within Boundary	0.6246	0.2221	0.7876
Overlapping with Game Object	0.6008	0.2766	0.7342
Visibility and Occlusion	0.6509	0.2051	0.7559
Object Scaling with Distance	0.6525	0.2936	0.7386
Orientation Consistency Relative to Gravity	0.6262	0.2677	0.7646
Varying Rotation	0.6000	0.2669	0.7287
Check Correct Object Instantiation	0.6267	0.3073	0.7596

To determine whether linguistic rephrasings themselves significantly affected stability, I analyzed the distribution of group stability scores shown in Figure 5.3. A Lilliefors test indicated that the data were non-normal ($p = 0.0010$), so non-parametric statistical testing was used. I then applied the Kruskal-Wallis test to each model to examine whether different MR rephrasings changed stability scores in a statistically significant way.

The resulting p-values were 0.4973 for Gemma, 0.4982 for StarCoder, and 0.4969 for CodeLlama. Since all values are above the conventional 0.05 threshold, the null hypothesis cannot be rejected. In other words, while models differ substantially in their overall stability levels, those levels are not significantly altered by the specific wording of the MR rephrasings.

This result has two implications. First, it suggests that the stability metric captures something meaningful about the models themselves rather than being dominated by arbitrary wording effects. Second, it indicates that response inconsistency is less about a model being highly sensitive to minor rephrasing and more about whether the model has a stable internal

pattern of reasoning for a given MR. Thus, the main distinction revealed by RQ1 is not that rephrasings destabilize the models differently, but that some models are simply much more stable than others.

5.4.2 RQ2: Can a Multi-Agent Debate Framework Improve Stability?

The second research question examines whether structured debate can improve the consistency of MR identification. The average stability rises to 0.8309, exceeding the performance of all three individual models.

The gains are not uniform across relations. *Overlapping with Game Object* achieves the highest stability after debate (0.8818), while *Varying Rotation* remains the least stable (0.7402). Even so, all relations benefit from the debate setting relative to weaker individual-model cases. This suggests that debate is particularly effective in cases where a single model may initially miss an important aspect of the transformation but can be corrected through counterargument.

Table 5.4 further characterizes how the debate system behaves. Across all decisions, the framework produced 51,353 *yes-yes* agreements and 120,058 *no-no* agreements, along with 28,780 inconclusive cases. At the same time, 36,618 mismatches remained unresolved in the sense that disagreement persisted at some stage of the process. This shows that debate improves consistency, but does not eliminate ambiguity entirely.

Figure 5.4 visualizes the post-debate distribution of stability scores. Compared with the single-model setting, the distribution shifts toward higher values, indicating that structured deliberation produces more consistent judgments overall. Taken together, these results answer RQ2 positively: multi-agent debate improves stability. However, the uneven gains

Table 5.4: Decision Type Distribution Across Multi-Agent Debate. Each decision is made per metamorphic relation, leading to a total of 33,687 × 7 evaluations.

Decision Type	**Count**
Yes-Yes	51,353
No-No	120,058
Inconclusive-Inconclusive	28,780
Mismatches	36,618

across MRs also indicate that some transformations remain difficult even under collaborative reasoning, suggesting room for further refinement in how debate is structured or adjudicated.

5.4.3 RQ3: To What Extent Does Stability Correlate with Correctness?

Although stability is useful as a robustness signal, it does not necessarily guarantee correctness. A model could be consistently wrong. To examine this issue, the third research question compares stability against human-labeled correctness.

To construct a correctness benchmark, the first and second authors independently labeled model predictions for 20 randomly selected code samples across all 7 MRs, resulting in 140 annotated instances. Each prediction was evaluated by examining the source code and determining whether the model's MR judgment aligned with the expected behavior. Inter-rater agreement, measured using Cohen's Kappa, was $\kappa = 0.73$, which indicates substantial agreement and suggests that the manual labels are reliable enough for comparative analysis.

Using these labels, I computed model accuracy as the proportion of predictions matching the human annotations. CodeLlama achieved the lowest correctness alignment at 52.3%, despite having the highest stability in RQ1. Gemma reached 61.5%, and StarCoder achieved 64.7%. The multi-agent debate system performed best at 81.2%, indicating that structured

deliberation improved not only consistency but also alignment with human judgment.

To quantify the relationship between stability and correctness, I computed Spearman's rank correlation coefficient (ρ) for each model. The results are shown in Table 5.5.

Table 5.5: Spearman's Correlation Between Stability and Correctness

Model	**Spearman's ρ**	**Correlation Strength**
CodeLlama	0.32	Weak
StarCoder	0.51	Moderate
Gemma	0.45	Moderate
Multi-Agent Debate	0.73	Strong

These results show that stability alone is not a sufficient indicator of correctness. CodeLlama is the clearest example: it is the most stable individual model, yet its stability correlates only weakly with correctness. This means that it often produces internally consistent judgments that are still wrong. StarCoder and Gemma show moderate relationships between stability and correctness, suggesting that consistency is somewhat informative but not decisive. The strongest relationship is observed in the debate framework, where higher stability is much more closely aligned with correctness.

The answer to RQ3 is therefore mixed. Stability is useful, but only under certain conditions. In the single-model setting, it should be interpreted cautiously. In the multi-agent setting, however, stability becomes a much more meaningful signal because deliberation appears to filter out some of the consistently incorrect reasoning that individual models produce.

5.5 Discussion

This chapter set out to determine whether LLMs can reliably identify metamorphic relations in mobile AR code and whether stability can serve as a useful evaluation lens in the absence

of large-scale ground truth. The results reveal three broader insights.

5.5.1 Stability Varies by Both Model and Relation

The first major finding is that stability is not a uniform property of LLMs. It varies substantially across models and, within each model, across metamorphic relations. Figure 5.5 illustrates this variation.

StarCoder and CodeLlama both exhibit noticeable dips for particular relations. For instance, StarCoder drops on *Object Scaling with Distance*, while CodeLlama weakens on *Overlapping with Game Object*. Gemma is unstable more broadly across nearly all relations. These patterns suggest that stability depends not only on general model capability but also on the kind of spatial or interactional reasoning required by the MR. Relations involving collision, scaling, or device-dependent spatial transformations appear to be more challenging than those with simpler behavioral cues.

This matters for AR software engineering because it indicates that LLM assistance should not be treated as uniformly reliable across all behavioral properties. A model that performs well on one class of AR relation may still struggle on another. As a result, practitioners should be cautious about deploying single-model pipelines for MR-related reasoning without understanding which kinds of relations the model handles well.

5.5.2 Debate Improves Reliability, but Not Equally Everywhere

The second major finding is that multi-agent debate produces more balanced and reliable reasoning. Compared with individual models, the debate framework reduces sharp fluctuations across MRs and yields consistently higher stability. Importantly, it also improves

correctness alignment, not just surface consistency.

This is a meaningful result for the broader dissertation because it supports the claim that LLMs are more useful in AR software engineering when their outputs are not taken at face value, but instead subjected to structured reasoning and validation. The debate mechanism acts as a lightweight form of internal review: one model can expose weaknesses in another's initial interpretation, and the final decision benefits from that confrontation.

At the same time, debate does not solve every problem. The remaining disagreement cases show that some MRs remain intrinsically difficult. In particular, transformations involving nuanced spatial semantics still generate persistent ambiguity. This implies that debate frameworks should not be viewed as complete replacements for domain knowledge or human oversight. Rather, they are best understood as mechanisms that improve reliability by reducing some, but not all, forms of model inconsistency.

5.5.3 Stability Is Useful, but Only When Interpreted Carefully

The third and most important insight is conceptual: stability is informative, but it is not equivalent to correctness. This distinction is especially important in software engineering tasks where direct ground truth is sparse or costly to obtain. Stability can reveal whether a model behaves consistently under controlled variation, which is valuable in its own right. However, the CodeLlama results show that a model may be stable for the wrong reasons. It may consistently apply an incorrect heuristic, leading to a high stability score but poor correctness alignment.

This suggests that stability should be treated as a complementary evaluation dimension rather than a standalone indicator of quality. In this chapter, its value becomes strongest when paired with deliberative mechanisms. Once multi-agent debate is introduced, stability

correlates much more strongly with correctness, indicating that consistency after challenge and refinement is more trustworthy than consistency from a single model alone.

More broadly, this finding has implications beyond AR. Many emerging software engineering tasks involving LLMs do not have clean gold-standard outputs at scale. In such cases, stability offers a practical way to assess whether model reasoning is robust to controlled perturbations. But this chapter also shows that such robustness must still be grounded by some form of validation, whether through human annotation, debate, or downstream behavioral checks.

5.5.4 Consensus Dynamics in Multi-Agent Debate

A closer look at the debate process also reveals how consensus is formed. Across the full set of 33,687 instances, 66.52% entered the debate phase because StarCoder and CodeLlama initially disagreed, while 32.46% reached immediate agreement. This indicates that disagreement is not an edge case but a common condition when applying LLMs to MR reasoning.

Among debated cases, the final outcome aligned with CodeLlama's original position 48% of the time and with StarCoder's original position 37% of the time. The remaining 15% ended inconclusively. These numbers suggest that CodeLlama's initial answers more often shaped the final decision, even though its standalone correctness was lower than that of StarCoder. This is an interesting result: influence within debate does not necessarily correspond directly to correctness in isolation. It may instead reflect rhetorical persistence or the kinds of arguments the adjudication mechanism was more likely to favor.

The disagreement rates also differ substantially by MR, as shown in Table 5.6. *Overlapping with Game Object* and *Orientation Consistency Relative to Gravity* generated the highest

disagreement rates, whereas *Varying Rotation* produced the fewest disagreements.

Table 5.6: Disagreement Cases Across Metamorphic Relations

Metamorphic Relation	**Disagreement Cases**
Overlapping with Game Object	28,209
Orientation Consistency Relative to Gravity	27,370
Check Correct Object Instantiation	26,458
Object Scaling with Distance	25,265
Raycast within Boundary	21,482
Visibility and Occlusion	11,916
Varying Rotation	8,574

These patterns reinforce the earlier interpretation that some AR relations are simply more difficult for LLMs to reason about. They also suggest that future debate-based systems may benefit from relation-specific prompting strategies or domain-sensitive adjudication criteria rather than a single generic debate policy.

5.6 Threats to Validity

Several threats to validity should be considered when interpreting the results of this chapter.

Internal Validity. The results depend on specific modeling and prompting choices, including the selection of StarCoder, CodeLlama, and Gemma, the prompt format, and the cosine-similarity-based stability metric. Different models, prompt structures, or response similarity measures could lead to somewhat different results. To mitigate this threat, I used the same prompt structure across models, applied systematic preprocessing to the source code, and defined stability through a consistent mathematical formulation rather than informal judgment.

Construct Validity. This chapter treats stability as consistency across rephrasings of the same MR. While this is a reasonable operationalization, it does not capture every aspect of reliable reasoning. A model may be stable but incorrect, as demonstrated by the CodeLlama results. To address this limitation, the chapter explicitly separates stability from correctness and includes a human-labeled analysis of their correlation rather than assuming that the two are equivalent.

External Validity. The study focuses on mobile AR repositories and on a fixed set of AR-specific metamorphic relations derived from prior work. As a result, the findings may not generalize directly to other immersive domains, other types of software, or other forms of behavioral properties. However, the evaluation framework itself is designed in a way that could be transferred to other domains where semantic consistency under transformation is important.

Conclusion Validity. The correctness analysis relies on a manually labeled subset of 140 instances. Although inter-rater agreement was substantial, the labeled sample remains relatively small compared with the full experimental corpus. The reported relationship between stability and correctness should therefore be interpreted as an informative empirical signal rather than a definitive characterization of all possible MR detection behavior.

5.7 Chapter Summary

This chapter examined whether LLMs can reliably detect metamorphic relations in mobile AR code and proposed stability as a new way to evaluate that capability. The results show that individual models differ substantially in their consistency, with CodeLlama emerging

as the most stable single model and Gemma the least stable. At the same time, the chapter demonstrates that stability alone is not sufficient: a model may be consistently wrong. To address this limitation, I introduced a multi-agent debate framework that improves both stability and correctness alignment, suggesting that structured deliberation is a promising mechanism for making LLM-based reasoning more dependable in AR software engineering.

Within the broader dissertation, this chapter serves two purposes. First, it validates that LLMs can participate meaningfully in MR-related reasoning, but only with careful evaluation. Second, it motivates the need for stronger LLM-centered testing workflows that do not merely generate outputs, but assess and refine them through structured mechanisms. This provides a direct bridge to the next stage of the dissertation, where LLMs are used not just to recognize behavioral properties, but to support richer testing and reasoning workflows for software correctness.

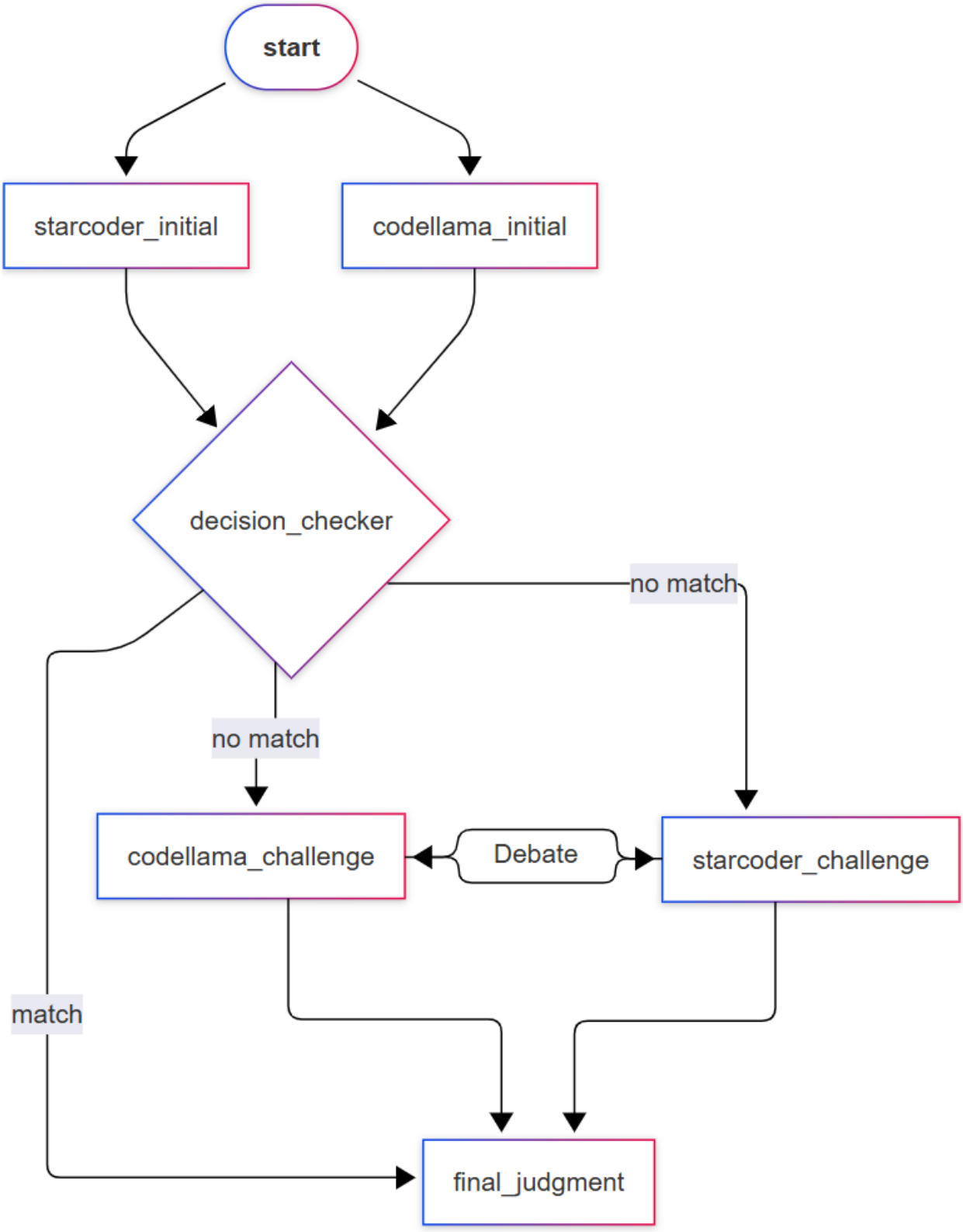


Figure 5.2: Multi-Agent Debate Workflow

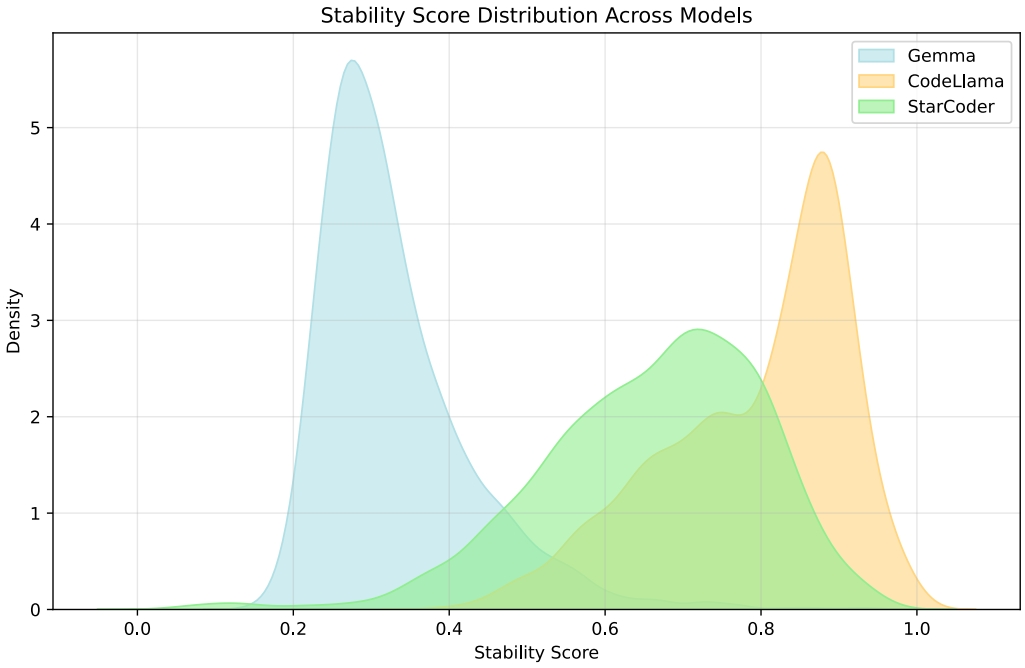


Figure 5.3: Group Stability Score Distribution of the three evaluated models

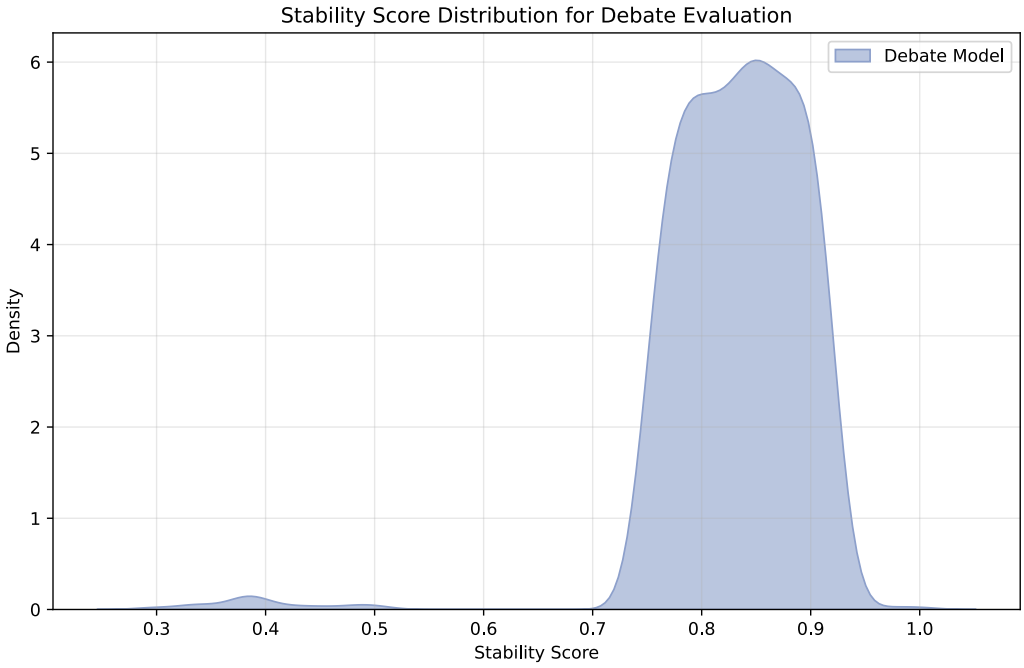


Figure 5.4: Group Stability Score Distribution of the multi-agent debate system



Figure 5.5: Stability scores across metamorphic relations for individual models and the debate framework

Chapter 6

Evaluating Property-Based Testing for LLM-Generated Code

6.1 Chapter Overview

Large language models (LLMs) are increasingly used for code generation, but evaluating the correctness of generated programs remains difficult. Most existing evaluations rely on benchmark-style unit tests, which provide a direct way to check whether a solution works on a predefined set of examples. However, passing such tests does not necessarily imply broader behavioral correctness. A generated solution may satisfy visible examples while still violating important constraints, edge cases, or invariants.

This chapter investigates whether property-based reasoning can provide a stronger evaluation lens for LLM-generated code. Figure 6.1 shows the overall workflow. Instead of checking correctness only against fixed examples, property-based testing (PBT) examines whether a program preserves more general behavioral requirements across a wider range of inputs. We study this question in two ways: first, by asking LLMs to infer correctness properties from natural-language programming tasks, and second, by using those properties both to guide generation and to evaluate the resulting code.

The chapter addresses the following research questions:

RQ1: To what extent can property-guided prompting help LLMs produce executable and behaviorally correct code?

RQ2: What differences emerge when LLM-generated code is assessed through conventional benchmark testing versus property-based testing?

To answer these questions, we evaluate StarCoder [71] and CodeLlama on HumanEval and MBPP. The study examines both the quality of inferred properties and the extent to which property-based evaluation exposes weaknesses that remain hidden under standard benchmark tests.

6.2 Methodology

Our methodology studies whether property-based reasoning can serve as a useful intermediate layer in LLM-based code generation. The workflow has two stages. First, given a natural-language programming problem, the model is asked to infer a set of behavioral properties that a correct implementation should satisfy. Second, those properties are fed back into the model as guidance for generating code. The resulting programs are then evaluated using both example-based testing and property-based testing.

6.2.1 Datasets

We conduct the study on two widely used Python code-generation benchmarks: HumanEval and MBPP. These datasets complement each other in how they represent correctness. HumanEval emphasizes behavioral specification through task descriptions and embedded assertions, whereas MBPP provides explicit executable unit tests. Using both benchmarks allows

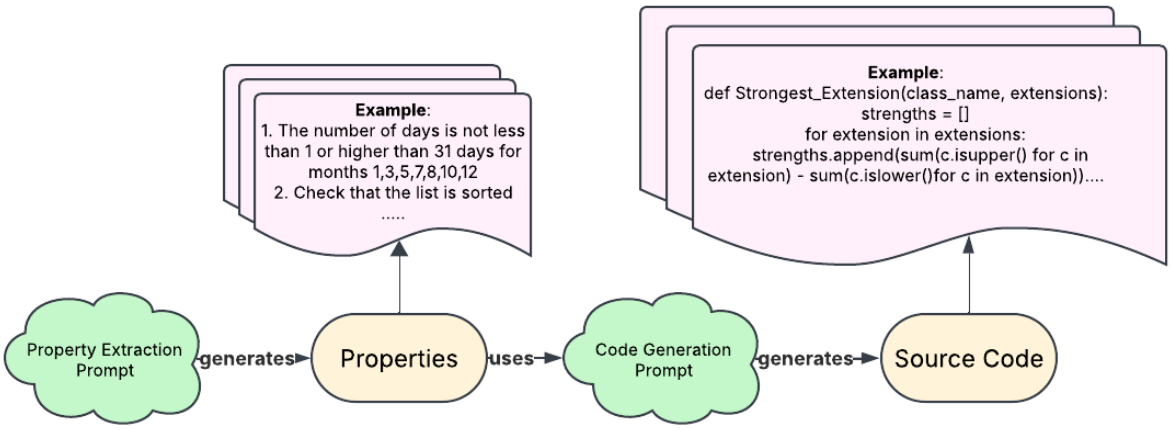


Figure 6.1: Pipeline of property generation and source code generation.

us to compare property-based reasoning with conventional test-based evaluation under different specification styles.

HumanEval contains 164 Python tasks, each with a problem description, target function signature, and example assertions in the docstring. MBPP contains 974 Python problems, each with a task description, reference implementation, and executable tests.

For detailed manual analysis, we randomly sampled 100 problems from each benchmark. The MBPP sample consists of shorter and structurally simpler programs on average, while the HumanEval sample contains somewhat longer solutions with richer control flow.

6.2.2 Prompting for Property Extraction and Code Generation

We use structured prompting in two stages: one for extracting behavioral properties from a task description, and another for generating code from those properties.

Prompt for Property Inference:

Objective: Read the programming task below and infer the core behavioral properties that a correct solution must satisfy.

Guidelines: - Focus only on correctness-related requirements.

- List the properties as concise bullet points.

- Include edge cases or invariants when they are implied by the task.

Task Description:

`<INSERT PROBLEM DESCRIPTION>`

Output:

Prompt for Property-Guided Code Synthesis:

Objective: Write a Python implementation that satisfies the behavioral properties listed below.

Guidelines: - Generate a correct and complete function.

- Ensure the implementation is consistent with all provided properties.

- Return only the code.

Behavioral Properties:

`<INSERT EXTRACTED PROPERTIES>`

Program:

6.2.3 Property-Based Extraction and Mapping

After the property inference stage, the extracted properties are manually examined. The first author labels each property according to whether it is valid, redundant, missing, or incorrect relative to the intended behavior of the problem. This step is necessary because the usefulness of property-guided generation depends on the quality of the inferred constraints.

Table 6.1: Representative correctness-property categories considered in our property-based analysis.

Property Category	**Interpretation**
Output Agreement	The returned result should match the expected behavior for valid inputs.
Return-Type Correctness	The produced value should have the required type (e.g., list, float, or boolean).
Boundary-Case Robustness	The implementation should behave correctly on inputs such as empty collections, zero, or negative values.
Ordering Requirement	When a list is returned, its elements should appear in the required order.
Input Preservation	The function should avoid mutating its inputs unless mutation is explicitly part of the specification.
Repeatability	Reapplying the function should preserve the same outcome when the task semantics require it.
Reversibility	When relevant, applying an inverse operation should recover the original value.
Algebraic Property	The implementation should preserve expected algebraic behavior such as associativity or commutativity when applicable.
Value Bounds	Outputs should remain within an expected numeric or logical range.
Duplicate Handling	Returned results should preserve or eliminate duplicates according to the task requirement.
Numerical Robustness	Floating-point computations should avoid unnecessary loss of precision.
String-Processing Consistency	String operations should follow the intended sensitivity rules, such as case-sensitive or case-insensitive behavior.

Table 6.1 summarizes the representative categories of behavioral properties considered in the analysis.

For each task, we sample ten candidate solutions, enabling evaluation under pass@1, pass@5, and pass@10.

6.2.4 Evaluation Strategy

We assess generated code under two validation settings.

Example-Based Evaluation. For MBPP, we execute the benchmark's provided unit tests. For HumanEval, we use the behavioral expectations expressed in the task descriptions and embedded assertions.

Property-Based Evaluation. We also construct property-oriented tests using the Hypothesis library.[1] These tests are derived from the manually validated property sets and are used to check whether generated programs preserve broader behavioral constraints across a range of inputs.

6.3 Results

6.3.1 RQ1: To what extent can property-guided prompting help LLMs produce executable and behaviorally correct code?

Table 6.2: Quality of inferred properties across MBPP and HumanEval (100 sampled problems), reported as percentages of useful, repeated, and omitted constraints.

Model	Benchmark	Useful (%)	Repeated (%)	Omitted (%)
StarCoder	MBPP	76	15	9
CodeLlama	MBPP	72	18	10
StarCoder	HumanEval	74	14	12
CodeLlama	HumanEval	70	17	13

[1] https://hypothesis.readthedocs.io/en/latest/

Table 6.3: Outcome of property-based verification on generated programs across MBPP and HumanEval

Model	**Benchmark**	**Complete Pass (%)**	**Partial Pass (%)**	**Violation (%)**
StarCoder	MBPP	52	30	18
CodeLlama	MBPP	48	32	20
StarCoder	HumanEval	49	29	22
CodeLlama	HumanEval	43	31	23

Table 6.2 shows that both models can infer useful behavioral properties from task descriptions, although the extracted sets remain imperfect. StarCoder produces a higher proportion of useful properties than CodeLlama on both benchmarks, while both models still exhibit repeated and omitted constraints.

These limitations carry over to downstream verification. As shown in Table 6.3, complete property satisfaction is achieved in roughly half of the sampled tasks, while the remaining cases are split between partial compliance and clear property violations. StarCoder performs slightly better than CodeLlama on both benchmarks, but neither model consistently satisfies all expected behavioral requirements.

Overall, the results suggest that property-guided prompting is helpful but incomplete. The models can recover meaningful behavioral constraints and use them to guide generation, yet the inferred property sets still contain gaps, and the resulting implementations do not always preserve all intended properties.

Table 6.4: Distribution of pass@k scores for StarCoder and CodeLlama on the MBPP benchmark

Model	Metric	Lowest	Average	Highest
StarCoder	pass@1	0.00	0.491	1.00
StarCoder	pass@5	0.01	0.521	0.74
StarCoder	pass@10	0.07	0.588	0.81
CodeLlama	pass@1	0.00	0.535	1.00
CodeLlama	pass@5	0.03	0.573	0.71
CodeLlama	pass@10	0.09	0.610	0.85

Table 6.5: Distribution of pass@k scores for StarCoder and CodeLlama on the HumanEval benchmark

Model	Metric	Lowest	Average	Highest
StarCoder	pass@1	0.00	0.460	1.00
StarCoder	pass@5	0.02	0.517	0.78
StarCoder	pass@10	0.06	0.533	0.86
CodeLlama	pass@1	0.00	0.500	1.00
CodeLlama	pass@5	0.06	0.520	0.75
CodeLlama	pass@10	0.08	0.594	0.82

6.3.2 RQ2: What differences emerge when LLM-generated code is assessed through conventional benchmark testing versus property-based testing?

Tables 6.4 and 6.5 show that both models benefit from multiple-sample generation under conventional benchmark evaluation. Average pass@k scores rise from pass@1 to pass@10 on both datasets, indicating that repeated sampling increases the chance of obtaining at least one benchmark-passing solution.

However, comparison with Table 6.3 shows that benchmark success does not fully translate to broader behavioral correctness. On both datasets, complete property compliance remains

lower than pass@10 performance. This indicates that programs that appear successful under available benchmark tests may still violate more general behavioral requirements.

The key difference between the two evaluation modes is therefore not simply numerical but conceptual. Benchmark testing measures success on predefined examples, whereas PBT checks whether implementations preserve broader behavioral rules across a wider input space. The latter exposes failures that standard benchmark tests may leave unobserved.

6.4 Discussion and Implications

The main finding of this chapter is that property-based testing provides a different and more behavior-oriented view of correctness than conventional benchmark evaluation. It does not replace unit testing, but it helps reveal cases where generated code satisfies visible examples without fully preserving the intended semantics of the task.

A first implication is that property extraction should not be treated as a trivial preprocessing step. The inferred properties directly shape both downstream generation and evaluation, yet the results show that these property sets often remain incomplete or repetitive. This means that property-guided generation is only as reliable as the intermediate behavioral constraints provided to the model.

A second implication is that benchmark success can overstate true correctness. The gap between pass@k and full property compliance shows that a generated solution may pass available tests while still violating broader requirements. This is especially important in LLM-generated code, where plausible-looking outputs can appear correct under narrow checks.

A third implication is that PBT makes failures more interpretable. Rather than showing only that a program fails, it helps identify what kind of behavioral requirement is violated, such

as ordering, robustness, or input preservation. This makes it useful not only for evaluation, but also for diagnosing where generated solutions remain weak.

Taken together, these findings suggest that the most reliable assessment framework for LLM-generated code should combine both evaluation modes: example-based testing for concrete functional validation and property-based reasoning for broader behavioral assurance.

6.5 Threats to Validity

Several threats to validity should be considered when interpreting the results of this chapter.

Internal Validity. The results depend on specific choices in prompting, model selection, sampling, and property-to-test mapping. Different prompt structures, different LLMs, or different property validation strategies could lead to somewhat different outcomes.

Construct Validity. This chapter treats extracted properties as an intermediate representation of intended program behavior. However, inferred properties are only an approximation of the original task specification, and some correctness aspects may not be fully captured by the property categories or mappings used here.

External Validity. The study is limited to two open-source LLMs, two Python benchmarks, and sampled tasks from each dataset. The findings may therefore not generalize directly to other languages, more complex software tasks, or newer models.

Conclusion Validity. The comparison between benchmark-style testing and property-based verification depends on manually validated property sets and derived executable

checks. Since this mapping involves human judgment, some subjectivity is unavoidable.

6.6 Chapter Summary

This chapter examined whether property-based reasoning can improve the evaluation of LLM-generated code and whether it reveals weaknesses that conventional benchmark testing fails to capture. The results show that LLMs can infer useful behavioral properties from task descriptions, but those properties remain incomplete and do not always guide generation toward full behavioral correctness.

The chapter also showed a consistent gap between benchmark-style success and full property compliance. This suggests that conventional example-based testing alone can present an overly favorable picture of correctness, whereas property-based testing provides a broader behavioral perspective. Within the broader dissertation, this chapter extends the central argument that software correctness often requires richer abstractions than fixed expected outputs alone.

Chapter 7

Repository-Aware Generation of Metamorphic Relations for Augmented Reality Applications

7.1 Chapter Overview

The preceding chapters established two premises that motivate this chapter. First, AR software is difficult to test because its behavior depends on physical context, sensor observations, camera motion, and spatial interaction rather than only on deterministic program input. Second, metamorphic relations (MRs) provide a useful way to express correctness in such settings because they describe how outputs should change when inputs or environmental conditions are transformed. However, the MRs studied earlier were defined manually or used as known relations for detection. This chapter moves from MR recognition to MR generation: it investigates whether new candidate relations can be discovered directly from real AR software repositories.

AR applications are a natural setting for this problem. They operate at the intersection of digital content, physical environments, mobile sensing, and application logic, enabling immersive experiences in domains such as surgical guidance, industrial support, and interactive

learning [10, 38, 62]. Unlike conventional software, AR systems must continuously reason about camera observations, device motion, and spatial alignment between virtual objects and the physical world [10, 18, 38]. This dependence on changing real-world conditions makes validation difficult because expected outputs often vary with illumination, tracking fidelity, viewpoint, and device orientation [18].

Metamorphic Testing (MT) addresses this difficulty by replacing exact expected outputs with behavioral constraints that relate source and follow-up executions [8, 25]. For example, if a camera moves toward a virtual object, the rendered object should appear larger; if the camera moves away, it should appear smaller. Such relational expectations can be encoded as MRs and used to validate AR behavior even when exact rendering results are unavailable [18]. The main challenge is discovering useful MRs at scale. Constructing meaningful relations requires domain knowledge [32], careful reasoning about input transformations, and an understanding of how those transformations can be translated into executable tests [73]. This challenge becomes more difficult in large repositories, where relevant behavior may be distributed across many classes and methods [68].

LLMs create an opportunity to automate parts of this process. Prior work shows that LLMs can support test creation and program reasoning [19, 51, 68]. However, most prompting strategies provide only local program views, such as individual functions or short snippets, which limits the model's ability to reason about repository-level behavior [39, 84]. Insufficient context can also lead to tests or specifications that are irrelevant, incorrect, or behaviorally uninformative [83, 98]. Recent repository-level approaches therefore provide structured context beyond isolated files [19, 47, 79], including hierarchical repository summaries that help LLMs reason about large codebases within context-window constraints [92]. However, different contextualizations can still produce overlapping, incomplete, or semantically incompatible MR candidates for the same target method [12, 85]. This makes MR generation not

only a candidate-production problem, but also a reconciliation problem. Debate-oriented refinement has shown promise for this type of reconciliation in other software engineering settings, including code generation [53] and software issue resolution [69].

This chapter studies whether such repository-aware context can improve MR generation for AR applications. The task differs from prior work on LLM-based MR identification [16]. In MR identification, the model decides whether a known relation applies to a given code snippet. In MR generation, no ground-truth MR is provided in advance; the model must derive candidate behavioral relations from program context. This changes the problem from classification to discovery and introduces additional challenges of coverage, quality, contradiction, redundancy, and refinement.

To address these challenges, this chapter introduces a repository-aware MR generation pipeline that organizes context at three levels: repository-level metadata, class-level semantic information, and method-level signatures. The pipeline then applies a deliberation-based refinement stage that compares candidate MRs, detects contradiction and redundancy, and consolidates the candidates into more coherent outputs. The study is guided by the following research questions:

- **RQ1:** How does repository-level contextual granularity affect the structural validity and coverage of LLM-generated metamorphic relations?
- **RQ2:** To what extent can structured deliberation reconcile competing MR candidates generated from different context configurations?
- **RQ3:** Can debate-refined MRs serve as executable test oracles for AR applications, and how does deliberation strategy affect their quality?

The results show that hierarchical repository context improves coverage, specificity, and

diversity while maintaining strong structural validity. They also show that simply adding more context is not enough: flat retrieval weakens source traceability relative to both the method-only and hierarchical settings, indicating that context organization matters as much as context volume. The deliberation mechanism resolves conflicts among MR candidates in 90% of cases, reduces the exact duplicate rate from 13.3% to 1.3%, and produces relations that are more precise and more sensitive to repository context. Finally, a manual oracle study shows that debate-refined MRs achieve strong validity (1.80/2) and testability (1.82/2), with 61.7% of curated outputs rated high-quality across all evaluated dimensions.

The main contributions of this chapter are as follows:

- An empirical investigation of how different levels of repository context influence the coverage, validity, and diversity of generated metamorphic relations for AR software, showing that contextual structure matters as much as contextual quantity.
- A deliberation-centered refinement framework that identifies contradictions and redundancies across candidate MRs and produces relations that are more coherent and more context-aware.
- An oracle-oriented evaluation of curated MRs, analyzing their validity, testability, and AR-specific usefulness as test specifications.

7.2 Approach

7.2.1 Context Configurations for MR Generation

Prior work on repository-scale code reasoning suggests that LLMs perform better when code is presented through structured contextual representations rather than isolated snip-

pets [47, 79, 92]. This is especially relevant for Unity-based AR systems, where behavior is rarely determined by a single method in isolation and instead emerges from interactions among classes, components, and framework-specific abstractions [13, 72]. Motivated by this, we design multiple context configurations to study how the amount and organization of repository information influence MR generation.

Our goal is not simply to provide more context, but to understand which form of context is most useful for producing high-quality metamorphic relations. To this end, we construct three prompting configurations that differ in both contextual scope and structure. Together, they form a controlled ablation over repository information, enabling us to analyze how increasing contextual richness affects the structural validity, behavioral specificity, and diversity of the generated MRs.

A0: Method-Only Context. A0 serves as the minimal baseline. Under this configuration, the LLM is given only the signature of the target method and no additional information about the enclosing class, related components, or overall repository. This setup tests how much behavioral reasoning the model can perform from the function interface alone. The advantage of this configuration is that it minimizes distraction from unrelated repository details. Its main limitation, however, is that the model must infer intent without access to the surrounding software structure, which restricts its ability to reason about behavior that depends on class-level semantics or cross-component interactions.

A1: Flat Repository Context. A1 expands the prompt by supplementing the target method with repository-level artifacts gathered from the codebase. These include the repository name, project type, Unity-related features referenced in the project, field declarations from classes, and additional class snippets selected using method-level similarity. All of this

information is inserted into the prompt as a single flat block, without explicitly representing the structural relationships among the retrieved elements. This configuration gives the model access to broader implementation signals than A0, but the context remains unorganized. As a result, it provides more information while offering limited guidance about how the retrieved artifacts relate to the target method.

A2: Hierarchical Repository Context. A2 introduces a context representation that explicitly mirrors the hierarchical structure of object-oriented software, which is common in AR development pipelines [46]. Instead of presenting repository information as one undifferentiated block, the prompt is organized into three levels. The repository layer provides global context, including the repository name, project type, and an LLM-generated summary of repository purpose. The class layer contributes class identity, inheritance information, an automatically generated class summary, and an indicator of whether the class functions as a Unity component. The method layer includes the target method signature together with related classes identified through a dependency graph. This organization is intended to expose both high-level architectural intent and lower-level implementation dependencies, allowing the model to reason about program behavior across multiple levels of abstraction.

7.2.2 Metamorphic Relation Generation Pipeline

We begin with Unity-based mobile AR repositories and identify target tuples of the form (r, c, m), where r denotes the repository, c denotes the class, and m denotes the method. For each target tuple, we query the language model under three alternative context configurations: A0, A1, and A2. The key distinction among these settings is not the generation objective itself, but the scope and organization of the context supplied to the model.

In A0, the model is exposed only to the target method signature, giving it access to the

most local form of information. In A1, the prompt is expanded with a flat collection of retrieved repository artifacts, including class fields, Unity-related features, and additional code snippets. In A2, the model receives a more explicitly organized representation that includes repository summaries, class summaries, inheritance structure, and dependency information. This progression enables us to examine not only whether more context helps, but also whether the *form* in which that context is presented affects the quality of the generated MRs.

For each prompted target method, the model outputs one or more candidate metamorphic relations. These candidates are represented using a fixed schema that includes fields such as `name`, `input_transformation`, `output_relation`, `target_class`, `target_method`, and `source`. The inclusion of `source` allows each generated relation to be traced back to the configuration from which it was produced, making it possible to later analyze how different contextualizations influence the resulting MR set.

After generation, all candidates associated with the same (r, c, m) tuple are collected into a shared pool and forwarded to the next reasoning stage. This two-stage design intentionally separates proposal from refinement. The first stage is responsible for surfacing a diverse set of plausible candidate relations under multiple contextual views, while the second stage is responsible for comparing, reconciling, and improving those candidates through structured reasoning.

7.2.3 Reasoning-Orchestrated Debate for MR Refinement

Figure 7.1 depicts the refinement stage used to reconcile the competing MR candidates generated under A0, A1, and A2. Once candidate relations have been produced for a given target tuple (r, c, m), they are brought together into a shared comparison set and passed to

a critique-driven deliberation stage.

Debate Input. For each target method, the refinement stage receives all candidate relations generated under the different context settings. Each candidate carries its proposed input transformation, expected output relation, and provenance label indicating whether it originated from A0, A1, or A2. Because these candidates are produced from different contextual views of the same method, they often vary substantially in precision, abstraction level, AR-specific grounding, and behavioral coverage.

Contradiction and Redundancy Analysis. We focus the critique stage on contradiction and redundancy because these two dimensions capture the most immediate relationships that can arise when several MR candidates are generated for the same target method: the candidates may either conflict with one another or restate essentially the same hypothesis [25, 128]. This makes them particularly appropriate for evaluating the purpose of the debate stage. Other qualities, such as structural validity, schema completeness, and field-level detail, are already assessed earlier at the level of individual MRs in RQ1. By contrast, the refinement stage is not primarily concerned with whether a single candidate is well formed, but with whether the set of candidates can be reconciled into a coherent and useful outcome. In this setting, contradiction represents unresolved disagreement among candidates, whereas redundancy indicates repeated claims that do not expand the testing hypothesis space. These two dimensions therefore directly reflect whether debate is functioning as a reconciliation mechanism rather than merely choosing one candidate arbitrarily.

- **Contradiction:** We treat two MRs as contradictory when they prescribe incompatible output behavior for similar or overlapping input transformations. For instance, one candidate may state that a transformation should preserve behavior, whereas another

may predict that the same transformation should alter the output. Detecting such clashes is necessary to maintain logical consistency in the final specification set [107].

- **Redundancy:** We treat two MRs as redundant when they express the same underlying behavioral expectation with only superficial wording differences. Such repetition does not increase testing power and instead reduces the diversity and practical usefulness of the resulting MR set [128].

Critique-Based Decision. After analyzing the candidate set for contradiction and redundancy, the judge issues one of three possible refinement outcomes:

- **KEEP_BEST**: retain the single candidate that most accurately and clearly represents the target behavior.
- **MERGE**: synthesize compatible information from multiple candidates into a single improved relation.
- **DISCARD_ALL**: reject the entire candidate set when the proposed relations are either mutually inconsistent or too weak to be useful.

As illustrated in Figure 7.1, this stage serves as a principled filtering and consolidation step, removing contradictory or duplicated relations before a final MR is produced.

Refined Output. The result of the debate stage is a refined MR together with an accompanying rationale and its source attribution. In practice, this output is usually more grounded in repository context and less repetitive than any one candidate considered in isolation.

Overall, the debate stage functions as a reconciliation layer that enforces coherence across competing MR candidates and removes unnecessary duplication. By explicitly reasoning about how candidates agree, differ, or overlap across context settings, the framework produces relations that are more consistent, more specific, and more suitable for downstream testing.

7.3 Evaluation Setup

This section describes the dataset, evaluation metrics, and analysis procedures used to investigate the research questions introduced in Section 7.1. Our evaluation focuses on mixed methods approaches to understand how repository-level contextual information influences the structural quality and diversity of generated metamorphic relations (RQ1), how reasoning orchestration through debate refines competing MR candidates (RQ2), and the validity and usefulness of generated MRs for AR applications (RQ3).

7.3.1 Dataset

We use an existing dataset of open-source, executable mobile AR repositories introduced in prior work [16]. The dataset consists of Unity-based mobile AR repositories written in C# and was collected from GitHub using AR-related keywords such as *"AR Foundation"*, *"ARCore"*, *"ARKit"*, and *"Augmented Reality"*.

From repositories in this dataset, we extract class-method pairs representing potential behavioral units for analysis. In total, the dataset contains 142 repositories and 5,167 unique class–method targets. For each target method, we generate metamorphic relation (MR) candidates using the three context configurations described in Section 7.2. Across all config-

urations, the generation process produced 14,916 candidate MRs, which form the basis for our experimental analysis.

7.3.2 LLMs Configuration

We use two open-weight instruction-tuned LLMs, as generation and deliberation impose different requirements. MR generation must scale across many repository-method instances, favoring an efficient model, whereas deliberation requires stronger reasoning to compare alternatives, resolve contradictions, and refine candidate MRs. We discuss the implications of this capability asymmetry in Section 7.5.

For MR generation, we use `Llama-3.2-3B-Instruct`, selected for efficient large-scale inference and strong instruction-following performance [81, 104]. We set `temperature`=0.3 and `max_new_tokens`=1024.

For refinement, we use `Qwen2.5-Coder-32B-Instruct`, leveraging its strong code reasoning ability to resolve contradictions, remove redundancies, and refine candidate MRs [49, 123]. We set `temperature`=0.3 and `max_new_tokens`=1024.

7.3.3 Structural Validity and Redundancy Metrics (RQ1)

To evaluate the quality of generated metamorphic relations (MRs), we adopt a set of complementary metrics capturing structural validity, specificity, diversity, and coverage. We design the structural validity metrics based on minimal criteria required for a usable MR, ensuring that each relation is well-formed, sufficiently descriptive, and actionable for defining input–output transformations [12, 25, 80]. In contrast, the remaining metrics are adapted from prior work on test generation and specification mining, where coverage, specificity, and

diversity are commonly used to assess the usefulness of generated artifacts [47, 79, 92].

Data-Driven Specificity Thresholds To avoid arbitrary heuristics when measuring textual specificity, we derive non-triviality thresholds from the pooled set of all generated MRs across A0, A1, and A2 (N = 14,916). For both the `input_transformation` and `output_relation` fields, we compute the 10th percentile (p10) character length using linear interpolation over the sorted distribution.

For a textual field f, non-triviality is defined as:

$$\text{NonTrivial}(f) = \begin{cases} 1 & \text{if } |f| \geq \text{p10} \\ 0 & \text{otherwise} \end{cases}$$

The resulting thresholds are 30 characters for `input_transformation` and 34 characters for `output_relation`. These thresholds are computed once and applied consistently across all approaches.

Structural Validity Criteria. We operationalize six binary criteria to assess whether a generated MR is structurally valid and usable for specifying testable input–output transformations. In particular, field completeness (C1), target validity (C4), and source traceability (C5) ensure that each MR can be mapped to a specific code location and generation setting. Non-triviality checks (C2–C3) enforce that both the input transformation and expected output relation contain sufficient detail to be operationalized as test assertions. Finally, the non-self-referential rationale constraint (C6) prevents degenerate cases where the model simply repeats the transformation or relation without meaningful justification.

Formally, each MR is evaluated against the following six binary criteria, where $C_i \in \{0, 1\}$:

1. **Field Completeness (C1):** All required fields (`id`, `name`, `input_transformation`, `output_relation`, `target_class`, `target_method`, `source`) must be present and non-empty.

2. **Input Non-Triviality (C2):** The `input_transformation` field satisfies the p10 threshold (≥ 30 characters).

3. **Output Non-Triviality (C3):** The `output_relation` field satisfies the p10 threshold (≥ 34 characters).

4. **Target Symbol Validity (C4):** Both `target_class` and `target_method` must be explicitly defined.

5. **Source Traceability (C5):** The `source` field must indicate the origin configuration (A0, A1, or A2).

6. **Non-Self-Referential Rationale (C6):** The `rationale` field must not duplicate the transformation or relation text.

Composite Structural Validity Score. To summarize structural quality, we compute a composite Structural Validity Score (SVS) as the average of the six criteria. The score ranges from 0 to 1.0, where 1.0 indicates that all structural constraints are satisfied.

$$\text{SVS}(mr) = \frac{1}{6}\sum_{i=1}^{6} C_i$$

We assign equal weight to each criterion to avoid introducing subjective bias about their relative importance. Since all criteria represent necessary (but not individually sufficient) conditions for a valid MR, equal weighting provides a neutral and interpretable aggregation that captures overall structural soundness without overfitting to any single aspect.

Redundancy Calculation We measure redundancy using exact and near-duplicate detection based on normalized MR signatures.

Exact duplicates are defined as MRs that have identical `input_transformation` and `output_relation` after normalization. We group MRs by this pair and count duplicates by keeping one instance per group and treating the remaining instances as redundant. The exact duplicate rate is computed as the proportion of such redundant instances over the total number of MRs.

Near duplicates are defined more loosely as MRs that target the same method and use the same `input_transformation`, but may differ in their `output_relation`. We group MRs using this relaxed signature and compute the near-duplicate rate using the same counting procedure.

Coverage Metrics. We measure coverage to quantify how broadly each approach explores the program space when generating metamorphic relations (MRs). In the context of MT, broader coverage increases the likelihood of identifying diverse and non-overlapping behavioral relations [12, 80].

- **Total MRs Generated:** The total number of MRs reflects the overall generation capacity of an approach and its ability to propose candidate behavioral relations.
- **Repository Coverage:** The number of unique repositories indicates how well the approach generalizes across different projects and codebases.
- **Class Coverage:** The number of distinct classes captures how broadly different components within a system are explored.
- **Class–Method Coverage:** The number of unique class–method pairs provides a fine-grained view of coverage, indicating how many specific program locations are

associated with generated MRs. To avoid ambiguity, methods are identified using `ClassName.methodName`.

Unique MR Rate. Finally, we compute the unique MR rate as:

$$\text{UniqueMRRate}(A) = 1 - \text{ExactDupRate}(A)$$

This metric reflects the proportion of structurally distinct metamorphic relations generated by each configuration.

7.3.4 Debate Outcome Analysis (RQ2)

To analyze how reasoning orchestration refines competing MR candidates, we examine the outcomes of the debate stage introduced in Section 7.2. We record the distribution of these decisions, identify which context configuration most frequently produces the winning MR, and analyze conflict signals such as contradictions or redundancies detected during debate. These measurements allow us to quantify how reasoning orchestration consolidates candidate relations into a smaller set of refined metamorphic relations.

7.3.5 Test Oracle Study (RQ3)

To evaluate whether debate-refined metamorphic relations (MRs) can serve as practical test oracles, we conduct a manual evaluation study on a curated subset of MRs. Starting from the full set of debate-refined MRs (N = 3,760), the first author manually reviewed all candidates and selected 141 MRs that capture AR-specific behaviors. The selection is based

Table 7.1: Evaluation Criteria for Test Oracle Study

Dimension	Key Question	Evaluation Criteria
Validity	Is the MR logically correct and meaningful?	(1) Transformation defines a clear input change; (2) Output relation correctly relates source and follow-up outputs; (3) Transformation and relation are logically consistent; (4) Violations indicate real faults
Testability	Can the MR be directly implemented as a test?	(1) Transformation is programmatically applicable; (2) Output relation is expressible as an assertion; (3) Terms are unambiguous; (4) No interpretation is required for implementation
AR Specificity	Does the MR reflect AR-specific behavior?	(1) Uses AR concepts (e.g., pose, alignment, occlusion); (2) Not trivially applicable to non-AR systems; (3) Targets AR-related methods or behaviors; (4) Reflects real-world interaction or spatial logic

on the semantic content of each MR, including the input transformation, output relation, and associated program context (target class and method).

We note that MR names are not used as a filtering criterion, as prior work has shown that surface-level identifiers are often unreliable indicators of semantic intent in generated artifacts [22, 117]. Instead, selection is guided by whether the MR reflects AR-specific behaviors such as spatial transformations, pose updates, or rendering consistency.

Evaluation Dimensions. We evaluate each MR along three dimensions: *validity*, *testability*, and *AR specificity*. These dimensions are grounded in prior work on test oracle design and specification quality, where useful oracles must be logically correct, executable, and contextually relevant [14, 70].

Scoring Scheme. Following standard practices in empirical software engineering for qualitative artifact evaluation, we adopt a 3-point ordinal scale (0–2) for each dimension [116]. Such coarse-grained scales are commonly used to balance annotation consistency and discriminative power in human evaluation studies [55, 102]. The scale definitions are operationalized using structured checklists, as summarized in Table 7.1.

Annotation Procedure. Three independent raters (Rater 1, Rater 2, and Rater 3) evaluated all 141 MRs using the predefined criteria. Each rater independently assigned scores across all dimensions without coordination. This process follows standard multi-rater evaluation protocols used in empirical software engineering studies.

Inter-Rater Agreement. To assess annotation reliability, we compute Fleiss' κ for multi-rater agreement and Krippendorff's α (ordinal) to account for disagreement magnitude. In addition, we report exact agreement and adjacent agreement (difference ≤ 1), which is appropriate for ordinal scales with skewed distributions. Pairwise weighted κ is also computed to analyze agreement between individual raters.

7.4 Result

7.4.1 RQ1: Effect of Context Granularity

Table 7.2 shows that hierarchical context (A2) achieves the strongest coverage, specificity, and diversity overall, Though the relationship between context volume and quality is not uniformly monotonic across all criteria.

Table 7.2: Effect of Context Granularity on Specificity and Diversity

Category	**Metric**	**A0**	**A1**	**A2**
Input Transformation Specificity	Mean Length	60.8	72.6	75.0
Input Transformation Specificity	Median Length	47	63	69
Input Transformation Specificity	% $\geq$ p10 Threshold	86.7	90.4	92.4
Output Relation Specificity	Mean Length	68.0	77.5	73.5
Output Relation Specificity	Median Length	57	72	70
Output Relation Specificity	% $\geq$ p10 Threshold	90.8	91.7	92.0
Diversity	Exact Duplicate Rate	18.9	14.8	13.3
Diversity	Near Duplicate Rate	10.8	7.2%	6.7
Diversity	Unique MR Rate	81.1	85.2	86.7

Coverage. A2 achieves the highest coverage across all dimensions, including repositories (142), unique classes (474), and class–method pairs (5,167), compared to A0 and A1. This indicates that hierarchical context enables the model to generalize across a broader set of program elements, beyond simply generating more MRs.

Structural Validity. All configurations maintain high structural validity (SVS $\approx$ 0.96–0.97), but the relationship between context and quality is not monotonic. A1, despite providing more raw context than A0, degrades source traceability to 96.25%—the only configuration falling below 100% on this criterion. This suggests unstructured flat retrieval introduces noise that disrupts schema adherence without adding behavioral signal. Hierarchical context (A2) recovers this and goes further, achieving the highest non-triviality rates for both input (92.43%) and output relations (92.00%), indicating that structured context enables more meaningful transformations rather than merely longer outputs.

Specificity and Diversity. As shown in Table 7.2, A2 improves input specificity (highest mean and median lengths, and 92.4% above the threshold) while maintaining competitive output specificity. In addition, A2 reduces redundancy (lowest exact and near-duplicate rates) and achieves the highest unique MR rate (86.7%), indicating more diverse and distinct

MRs.

7.4.2 RQ2: Effect of Multi-Agent Debate

Table 7.3: RQ2: Conflict Identification and Resolution in Multi-Agent Debate ($N = 4496$)

Category	**Metric**	**Count**	**Rate**
Conflict Signals	Contradictions detected	3550	79.0%
Conflict Signals	Redundancies detected	2301	51.2%
Conflict Signals	No issues detected	442	9.8%
Resolution Decisions	MERGE	1919	42.7%
Resolution Decisions	KEEP_BEST	1841	40.9%
Resolution Decisions	DISCARD_ALL	736	16.4%

Table 7.3, Table 7.4, and Table 7.5 summarize the role of multi-agent debate in identifying conflicts and improving MR quality.

Conflict Identification and Resolution. As shown in Table 7.3, conflicts are prevalent among candidate MRs, with contradictions detected in 79.0% and redundancies in 51.2% of cases, while only 9.8% exhibit no issues. This highlights that independently generated MRs frequently disagree or overlap. The debate framework actively resolves these conflicts through structured decisions, primarily via MERGE (42.7%) and KEEP_BEST (40.9%), with DISCARD_ALL applied in 16.4% of cases. This indicates that the majority of candidate sets require refinement rather than direct acceptance.

Preference for Context-Rich MRs. Table 7.4 shows that the debate process overwhelmingly favors MRs generated with hierarchical context. A2 accounts for 88.2% of final selections, while A1 and A0 contribute minimally (1.2% and 0.2%, respectively). Additionally, 9.3% of cases result in merged outputs, suggesting that combining complementary

Table 7.4: RQ2: Winning Source Distribution After Debate ($N = 3760$)

Source	Count	Rate
A2 (Hierarchical)	3316	88.2%
MERGED	348	9.3%
A1	47	1.2%
A0	9	0.2%

information across sources can further improve MR quality. These results demonstrate that richer contextual information consistently leads to more reliable and preferred MRs.

Impact on MR Quality. The end-to-end impact of debate is shown in Table 7.5. Debate-refined MRs are more detailed, with higher average and median lengths for both input transformations (89.3 / 83.0) and output relations (101.2 / 95.0) compared to all individual configurations. At the same time, redundancy is substantially reduced, with the exact duplicate rate dropping from 13.3% (A2) to 1.3%. Although the total number of MRs decreases to 3,760, this reflects effective filtering and consolidation rather than loss of useful information.

Table 7.5: End-to-End Impact of Debate on MR Quality

Metric	A0	A1	A2	Debate-Refined
Total MRs	4046	3866	7004	3760
Avg Transformation Length	60.8	72.6	75.0	**89.3**
Median Transformation Length	47.0	63.0	69.0	**83.0**
Avg Relation Length	68.0	77.5	73.5	**101.2**
Median Relation Length	57.0	72.0	70.0	**95.0**
Exact Duplicate Rate	18.9%	14.8%	13.3%	**1.3%**

7.4.3 RQ3: Generated MR Practicality

To examine the suitability of generated MRs for testing real-world AR programs, we conducted a manual oracle study.

Table 7.6: Quality of Debate-Refined MRs

Dimension	Mean	SD	Majority High	Consensus High
Validity	1.80	0.35	86.5%	67.4%
AR Specificity	1.68	0.45	82.9%	48.2%
Testability	1.82	0.29	88.0%	63.8%

Inter-Rater Reliability. Three raters independently evaluated 141 debate-refined MRs on validity, AR specificity, and testability. Overall agreement is in the slight-to-fair range, with Fleiss' κ values of 0.33 for validity, 0.22 for AR specificity, and 0.19 for testability. However, these values should be interpreted together with the high adjacent agreement rates of 94.3%, 95.7%, and 96.5%, respectively, indicating that disagreements were usually limited to a one-level difference on the 0–2 scale. Pairwise weighted agreement reveals a clear structure: Rater 1 and Rater 2 are strongly aligned across all three dimensions ($\kappa_w = 0.73$, 0.73, and 0.58), whereas agreement involving Rater 3 is much lower, especially for testability ($\kappa_w = -0.01$ between Rater 1 and Rater 3). This pattern suggests that the lower multi-rater coefficients are driven less by random disagreement and more by a systematic difference in rating strictness, particularly for AR specificity and testability.

Quantitative Findings. Table 7.6 shows that validity and testability are the strongest dimensions, with mean scores of 1.80 and 1.82, and majority-high rates of 86.5% and 88.0%, respectively. This indicates that most debate-refined MRs are both logically sound and sufficiently well-specified for human raters to judge them as directly implementable as test conditions without requiring additional interpretation. AR specificity is lower and more variable (mean = 1.68, SD = 0.45), which is consistent with the fact that the curated set spans both AR-core behaviors and application-layer logic. Even so, 82.9% of MRs receive majority-high AR-specificity scores. Across all three dimensions, 87 of 141 MRs (61.7%) satisfy the high-quality criterion, and 35 (24.8%) achieve top-tier quality across all dimensions.

Effect of Deliberation Strategy. The debate strategy also affects oracle quality. MRs produced through MERGE outperform KEEP-BEST on every dimension, with a higher composite score (1.81 vs. 1.73), higher validity (1.85 vs. 1.77), higher AR specificity (1.70 vs. 1.66), and higher testability (1.87 vs. 1.78). This indicates that synthesizing information from competing candidates yields stronger oracle specifications than simply retaining a single candidate.

7.5 Discussion

Our findings offer several broader lessons for LLM-based MR generation, AR testing, and the practical use of metamorphic testing. These lessons are not limited to the specific pipeline studied here; they also speak to how repository-scale context and multi-candidate reasoning should be handled in future LLM-assisted testing systems.

7.5.1 Implications for LLM-Based MR Generation

A key takeaway from our results is that repository context does not help uniformly. The common assumption that additional context will naturally improve LLM reasoning is not supported by our findings. Instead, the utility of context depends strongly on how it is structured and how well that structure aligns with the underlying organization of the software being analyzed.

Context Configurations. One of the clearest findings is that flat retrieval (A1) can be worse than both a minimal baseline and a more structured alternative. In our results, A1 lowers source traceability below both A0 and A2, dropping from 100% to 96.25%. This decline is important because it shows that merely injecting more repository content into the

prompt does not guarantee better behavioral reasoning. In this case, the additional snippets retrieved under the flat setup often introduce loosely connected information that interferes with schema adherence without actually improving the model's understanding of method behavior. The problem is therefore not lack of context, but lack of organization.

By contrast, hierarchical context (A2) restores source traceability and improves other desirable qualities at the same time. It yields more specific input transformations (92.4% non-trivial transformations versus 86.7% for A0) and greater diversity in the resulting MR set (86.7% unique relations versus 81.1%). These improvements suggest that the model benefits not simply from seeing more artifacts, but from seeing them in a representation that preserves software structure. When repository-, class-, and method-level information are distinguished explicitly, the model can reason over relationships rather than over an undifferentiated block of text.

This observation has implications beyond MR generation. Many repository-level LLM tasks, including code search, bug localization, and specification mining, face the same basic trade-off: the model needs enough context to reason meaningfully, but too much poorly structured context can obscure the relevant signals [79, 92]. Our ablation indicates that the decisive factor is not only context quantity but contextual form. A hierarchical representation preserves relationships among repository layers and provides the model with a more interpretable behavioral landscape. For future LLM-based MR generation systems, context design should therefore be treated as a primary modeling decision rather than as a retrieval implementation detail.

Debate-Based Refinement. A second major implication is that MR generation should be treated as an inherently multi-perspective reasoning problem. The candidates produced under A0, A1, and A2 frequently disagree with one another, and they also frequently restate overlapping ideas. In our experiments, contradictions appear in 79.0% of cases and

redundancies in 51.2% of cases. This indicates that no single contextual view is sufficient to capture the full behavioral surface of a target method. Even when a configuration produces valid relations, those relations may still be incomplete, overly generic, or blind to other plausible behavioral hypotheses surfaced under a different context setting.

This has an important methodological consequence: refinement is not merely a polishing stage appended to generation. It is a structurally necessary stage in any multi-context pipeline. Without a reconciliation mechanism, the system produces candidate sets that are difficult to trust because they contain both conflict and duplication. Our results show that the debate mechanism performs precisely this missing role. It reduces exact duplicates from 13.3% to 1.3%, and it consistently either selects the most contextually grounded relation or synthesizes a stronger one from multiple partial candidates. In the oracle study, merged outputs achieve a higher composite score than single-best selections (1.81 versus 1.73), indicating that deliberative synthesis is often more effective than simple ranking.

More broadly, these findings suggest that LLM-based MR generation should not be conceptualized as a one-shot prompting problem. Instead, it is better understood as a staged reasoning process in which multiple context-sensitive hypotheses are generated and then reconciled. This view is especially important for repository-scale software, where behavior is distributed across interacting components and no single prompt perspective is likely to capture all relevant semantics. In such settings, debate is valuable not because it makes the model "think harder" in a vague sense, but because it explicitly creates a mechanism for comparing, filtering, and combining competing behavioral interpretations.

Taken together, the results of RQ1 and RQ2 point to two design principles for future LLM-based MR generation systems. First, repository context should be structured in a way that preserves architectural relationships rather than flattened into a single retrieval block. Second, candidate reconciliation should be built into the pipeline as a core reasoning stage

rather than treated as optional post-processing. These two principles work together: structured context broadens the quality of candidate generation, and debate-based refinement converts that broader candidate space into a coherent and usable MR set.

7.5.2 Implications for AR Testing

Existing studies have shown that testing AR software remains difficult in practice, both because of the dynamic nature of AR behavior and because developers often struggle to reason about correctness under changing environmental conditions [17, 62]. Our results suggest that the value of generated MRs is not limited to acting as abstract behavioral descriptions. They can also serve as concrete testing assets when translated into executable specifications.

To make this practical connection explicit, we include a motivating example that shows how a generated MR can be reformulated as a property-based testing (PBT) specification. In this interpretation, the MR is no longer viewed only as a semantic statement about program behavior; instead, it becomes a reusable testing rule defined over transformed inputs and the corresponding expected output relation. This shift is important because it shows how automatically generated MRs can move beyond analysis and become part of the validation workflow itself.

In that example, a perturbation applied to coordinates in GL space is translated into an expected change in world-space position while camera parameters and depth remain fixed. The resulting property captures a relational expectation that can be checked repeatedly across generated inputs, rather than through a single hand-crafted example. In this way, the MR becomes an executable testing abstraction rather than only a descriptive specification.

This example highlights a broader implication for AR testing. Debate-refined MRs can

provide a bridge between high-level behavioral reasoning and operational test design. For AR systems, this is especially useful because many important behaviors are relational and environment-dependent, making them difficult to validate with conventional fixed-output assertions. By expressing such behaviors as reusable properties, MR-based specifications create a path toward more systematic and automatable testing of AR codebases.

7.5.3 Implications for Practitioners

Although the overall results are encouraging, practitioners should interpret them with appropriate caution. In particular, 38.3% of the evaluated MRs do not satisfy our high-quality threshold simultaneously across all three evaluation dimensions. This points to two practical considerations that are important for real-world adoption.

Human-in-the-loop. One practical lesson is that structural validity alone is not sufficient to guarantee immediate usability. An MR may be well formed in schema terms and still remain too broad, underspecified, or difficult to instantiate directly in a testing workflow. This issue is especially visible for application-layer behaviors where AR-specific semantics are less explicit. Among the evaluated dimensions, AR specificity exhibits the greatest variation (mean 1.68, SD 0.45), indicating that the pipeline is most dependable when the target behavior is closely tied to core AR mechanisms such as pose propagation, spatial transformation, or rendering consistency. It is less consistently reliable for higher-level application logic, where the behavioral connection to AR semantics may be weaker or more indirect. For practitioners, this means that generated outputs are best treated as prioritized candidate specifications rather than final testing artifacts. In practice, a lightweight review step by a developer or domain expert is likely to remain necessary before direct adoption.

Model asymmetry and generalizability. A second implication concerns the design choices behind the pipeline itself. Our system deliberately combines a smaller generation

model (Llama-3.2-3B) with a larger deliberation model (Qwen2.5-Coder-32B) in order to balance generation scalability with stronger downstream reasoning. While this pairing is effective in practice, it also raises an important interpretive question: the strong preference of the judge for A2-generated candidates (88.2%) may partly reflect a bias toward the hierarchical prompt structure used in that condition. At the same time, two observations argue against reducing the result to that explanation alone. First, the `MERGE` cases (9.3%) consistently outperform `KEEP_BEST` outcomes even though merging partially diffuses the direct A2 signal. Second, the independent human oracle study provides additional support that the refined MRs are genuinely stronger, rather than merely aligned with the judge model's preferences. Even so, future studies should examine deliberation using models of more closely matched capability so that the contribution of context structure can be separated more clearly from the contribution of model capacity.

A related issue concerns external validity. Although the pipeline itself is not specific to a single application domain, the empirical evaluation is grounded entirely in 142 Unity-based C# repositories. This means that the current evidence supports repository-aware MR generation most directly for AR software implemented in that ecosystem. Whether the same gains carry over to other settings—for example, robotics software, embedded systems, or other sensor-driven platforms—remains an open question that future work should address. For practitioners outside Unity-based AR development, the present results therefore provide a promising signal, but not yet a guarantee of direct transferability.

7.6 Chapter Summary

This chapter presented a repository-aware pipeline for generating and refining metamorphic relations from AR repositories using structured context and reasoning-based deliberation.

The results show that hierarchical context improves MR coverage and diversity, while debate-based refinement resolves conflicts and substantially reduces redundancy. The human oracle study further shows that many refined MRs are valid, testable, and AR-specific enough to function as practical test oracle specifications.

Overall, this chapter extends the dissertation from detecting known MRs to discovering new ones. It shows that LLMs can support correctness reasoning for AR software when they are given structured repository context and when their outputs are refined through explicit deliberation. In the broader thesis, this contribution completes the progression from empirically identifying AR testing challenges, to defining behavioral correctness relations, to using structured AI workflows to discover and refine such relations at repository scale.

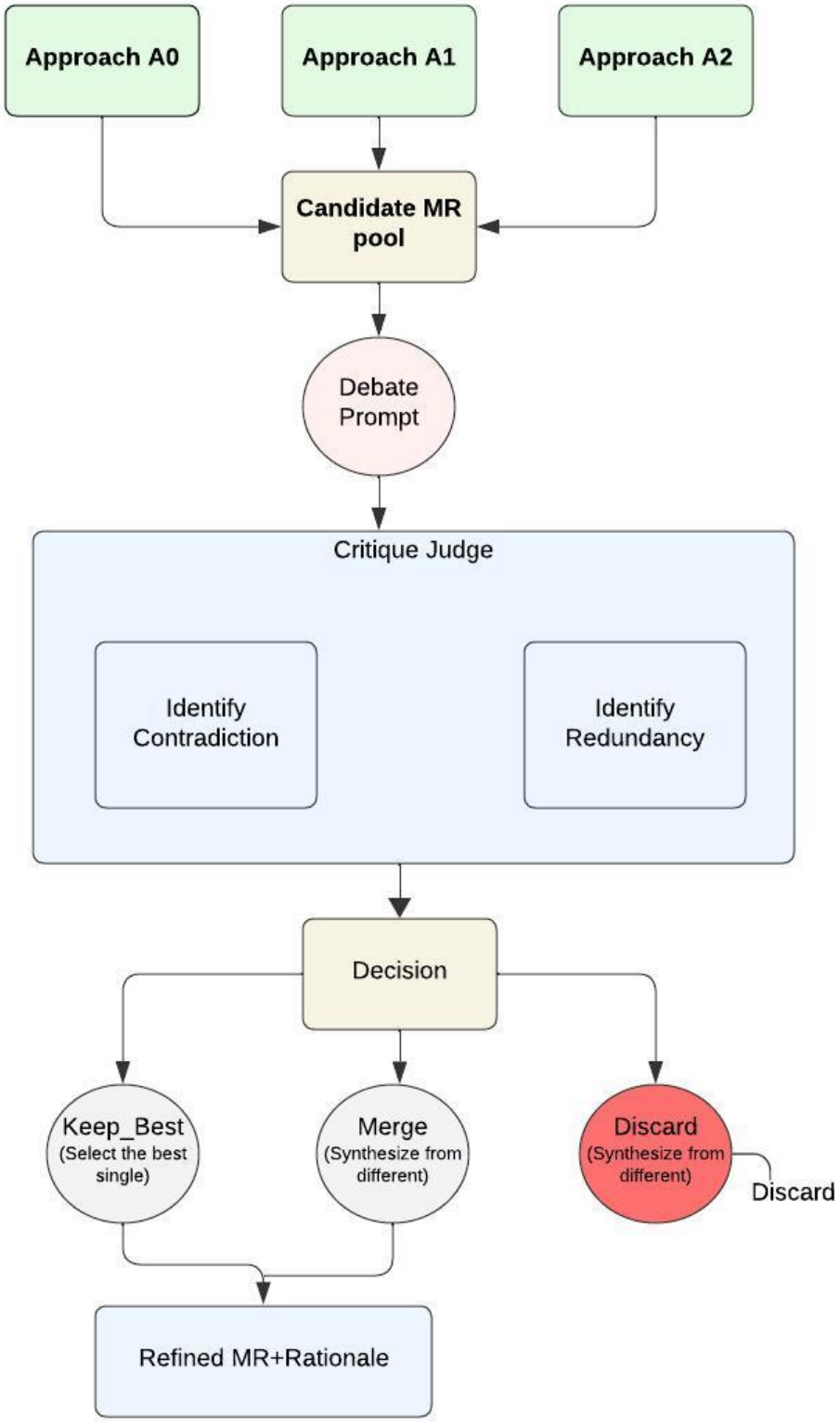


Figure 7.1: Debate-based MR refinement workflow. Candidate MRs generated under A0, A1, and A2 are aligned by target method, assembled into a debate prompt, evaluated by a critique/judge model, and converted into a final decision: `KEEP_BEST`, `MERGE`, or `DISCARD_ALL`.

Chapter 8

Future Work

Although this dissertation advances software engineering support for AR through empirical study, metamorphic testing, and LLM-assisted reasoning, it also points to a broader research agenda that remains open. The central outcome of this work is not that the problems of immersive software engineering have been fully solved, but that they can be approached in a more systematic way when developer practice, behavioral testing, and structured AI reasoning are brought together. Future work should therefore focus on deepening this integration and extending it toward more complete, scalable, and deployable support for immersive systems.

One important next step is to broaden the behavioral scope of the testing framework developed in this dissertation. The metamorphic relations introduced here capture several foundational properties of mobile AR, including scaling, occlusion, overlap prevention, instantiation, and gravity-relative orientation. However, immersive systems increasingly involve more complex forms of interaction that were outside the present scope. Future work should investigate how relation-based testing can be extended to collaborative multi-user experiences, multimodal interaction, spatial audio, gesture-centric interfaces, haptic feedback, and adaptive behaviors that evolve in response to user context. Such an extension is necessary if metamorphic testing is to mature from a promising AR-specific approach into a broader validation methodology for immersive software.

A related direction is to move beyond relation definition toward fully executable testing

workflows. In this dissertation, metamorphic relations function primarily as behavioral specifications that help reason about correctness in environments where conventional test oracles are weak. The next stage is to operationalize these specifications automatically. This includes generating source and follow-up test cases from metamorphic relations, translating them into property-based assertions, incorporating them into simulation-based or device-level execution environments, and embedding them within continuous integration pipelines. Advancing in this direction would connect the conceptual contribution of this dissertation more directly to everyday software engineering practice, allowing behavioral relations to function not only as analytical tools but also as executable testing assets.

Future work should also examine how the ideas developed here generalize beyond the mobile AR setting that served as the initial foundation of this dissertation. Mobile AR provided a practical starting point, but the software engineering challenges identified in this work are not confined to that platform alone. Head-mounted AR, VR, and mixed reality systems introduce additional complexities in sensing, rendering, user embodiment, latency, and interaction design. These systems may require new categories of metamorphic relations and different forms of environmental abstraction. Extending the present framework across platforms will therefore be essential for determining which aspects of the proposed methods are domain-independent and which must be specialized to particular immersive technologies.

The role of large language models also remains open for further development. This dissertation showed that LLMs can assist with metamorphic-relation reasoning, but also that such support is only trustworthy when stability and correctness are examined together. Future work should continue exploring how LLMs can be used not only to identify known metamorphic relations, but also to discover new ones, refine ambiguous ones, and connect them to implementation-specific context. In particular, repository-aware analysis offers a promising direction because correctness relations in real software often depend on architectural intent,

class responsibilities, API usage, and interaction among multiple files rather than isolated methods alone. Enriching LLM reasoning with hierarchical repository context, runtime observations, and project-level semantics may enable more meaningful discovery of correctness relations than prompt-based inspection of individual code fragments.

At the same time, the multi-agent perspective introduced in this dissertation invites a larger question about how AI should participate in software engineering tasks where uncertainty is unavoidable. The findings suggest that debate is useful not simply because it aggregates multiple answers, but because it creates a mechanism for surfacing contradiction, reducing redundancy, and refining reasoning under ambiguity. Future work should investigate richer multi-agent structures in which different agents perform distinct software engineering roles, such as specification extraction, test generation, critique, adjudication, and traceability analysis. This would move beyond the present use of debate as a corrective mechanism and toward a more general agentic framework for behavioral software analysis.

Another important direction is methodological. One of the recurring themes of this dissertation is that traditional notions of evaluation are often insufficient in domains where ground truth is incomplete, unstable, or expensive to obtain. The stability-based framework proposed here is one response to that problem, but it should not be the last. Future work should explore broader evaluation models for LLM-assisted software engineering that combine stability with correctness, interpretability, calibration, and human trust. Such work would be especially valuable in AR and related domains, where developers may rely on AI assistance in situations that are difficult to validate independently. Strengthening evaluation will therefore be just as important as improving generation or reasoning quality.

The connection between metamorphic testing and property-based testing also deserves deeper investigation. Across this dissertation, a common idea emerges: correctness in complex systems is often better expressed through relations, invariants, and behavioral constraints than

through isolated example-based outputs. Future work should build a more unified view of these techniques by studying how metamorphic relations can be transformed into property schemas, how extracted properties can suggest new metamorphic relations, and how both can be used together to validate software that evolves under uncertain conditions. A stronger synthesis of these approaches may ultimately contribute to a more general framework for specifying and checking correctness in domains where direct oracle construction is difficult.

Finally, the long-term significance of this dissertation depends on whether the proposed ideas can shape real developer practice. For that reason, future work should not focus only on technical refinement, but also on adoption, usability, and workflow integration. It will be important to understand how developers interpret automatically generated metamorphic relations, how much justification or explanation they need before trusting AI-assisted outputs, and how these tools affect the speed and quality of debugging, testing, and maintenance in practice. Human-centered studies of this kind would help ensure that the methods proposed here remain aligned with the realities of software engineering work rather than developing only as research abstractions.

In closing, the future work opened by this dissertation is best understood as an effort to build a more mature software engineering foundation for immersive and similarly hard-to-test systems. The broader challenge is not merely to create better AR tests or more capable AI tools, but to develop ways of reasoning about software behavior that remain rigorous even when environments are dynamic, outputs are context-dependent, and correctness cannot be fully enumerated in advance. The work presented here establishes one path toward that goal. Future research can extend it by making behavioral specifications more expressive, testing pipelines more automated, AI reasoning more dependable, and software engineering support more responsive to the realities of emerging interactive systems.

Chapter 9

Conclusion

In this dissertation, I argued that software engineering for AR systems requires a fundamental shift in how correctness, testing, and developer support are approached. Unlike traditional software, AR applications operate in environments shaped by physical context, sensor input, spatial interaction, and continuous change. These characteristics make immersive systems difficult to design, validate, and maintain using conventional software engineering assumptions. The central claim of this dissertation is that improving the reliability of AR software requires three things working together: an empirical understanding of the challenges developers actually face, testing methods that can reason about behavior without depending on fixed expected outputs, and AI-assisted techniques that can support this reasoning in a scalable and systematic way.

The dissertation began by grounding this argument in the lived experiences of core AR developers. That study showed that the difficulty of immersive software engineering is not limited to implementation complexity alone, but extends across the full software development lifecycle. Developers struggle with designing immersive experiences that remain scalable across platforms, implementing interactions that behave naturally in three-dimensional environments, and testing systems whose outputs depend heavily on context, environment, and user behavior. These findings established that AR development is not simply another application domain for existing software practices. It is a domain whose technical and testing problems are shaped by uncertainty, variability, and interaction with the physical world.

Building on that foundation, this dissertation showed that metamorphic testing provides a meaningful way forward for AR applications. Instead of requiring precise expected outputs in situations where such outputs are difficult or impossible to specify, metamorphic testing allows correctness to be expressed through behavioral relations that should continue to hold as inputs, viewpoints, or environmental conditions change. By identifying AR-specific metamorphic relations such as scaling with distance, occlusion behavior, overlap prevention, and gravity-relative orientation, this work demonstrated that many important properties of immersive systems can be tested through transformations rather than absolute answers. This is a significant step because it reframes AR testing around the kinds of invariants that matter in real use, rather than forcing the domain into testing assumptions it does not naturally satisfy.

The dissertation then examined the role of large language models in supporting this form of reasoning. Because AR software often lacks clear ground truth, evaluating AI assistance in this space cannot depend only on traditional accuracy measures. For that reason, this work introduced stability as an important lens for understanding whether models reason consistently about metamorphic relations across different formulations. The results showed that while code-focused models can be relatively stable, stability alone is not enough: a model may be consistently wrong. The introduction of multi-agent debate addressed this limitation by showing that structured interaction among models can improve both consistency and correctness alignment. This contribution suggests that LLMs are most useful in AR software engineering not when they act as isolated oracles, but when they participate in explicit reasoning and validation processes.

The broader implication of this dissertation is that AR software engineering benefits from moving away from fixed-answer validation and toward relation-based, property-based, and debate-supported forms of reasoning. This same idea also appears in the study of property-

based testing for LLM-generated code, where behavioral constraints revealed weaknesses that ordinary unit tests could miss. Across these contributions, a common theme emerges: reliable software engineering for complex and uncertain systems requires methods that reason about what must remain true under variation, not only what output should appear for a single input. That principle is especially important for immersive software, where variation is not an exception but a defining feature of the domain.

Taken together, the works in this dissertation establish a progression from understanding the problem, to designing domain-appropriate testing mechanisms, to exploring how intelligent systems can assist in scaling those mechanisms. The dissertation therefore contributes both practical techniques and a broader research perspective. It shows that empirical evidence should guide the design of testing methods, that metamorphic relations offer a strong foundation for validating AR behavior, and that AI can play a meaningful role when its reasoning is subjected to structured evaluation rather than accepted at face value.

In closing, this dissertation contributes to a larger vision of software engineering for immersive systems: one in which correctness is treated as contextual, behavioral, and evolving rather than fixed and easily enumerable. AR systems demand methods that can accommodate uncertainty without surrendering rigor. By combining empirical study, metamorphic testing, property-based reasoning, and multi-agent LLM support, this work takes a step toward that goal. It shows that the path to more trustworthy AR software is not to simplify the domain to fit conventional methods, but to develop software engineering techniques that are expressive enough to match the domain itself.

Bibliography

[1] [n. d.]. 10 Usability Heuristics for User Interface Design. "https://www.nngroup.com/articles/ten-usability-heuristics/".

[2] [n. d.]. 2024 Stack Overflow Developer Survey. https://survey.stackoverflow.co/2024/ai#sentiment-and-usage-ai-select.

[3] [n. d.]. Microsoft Hololens. "https://www.microsoft.com/en-us/hololens/".

[4] [n. d.]. Oculus Quest. "https://www.oculus.com/experiences/quest/".

[5] Ahmad Abdellatif, Diego Costa, Khaled Badran, Rabe Abdalkareem, and Emad Shihab. 2020. Challenges in Chatbot Development: A Study of Stack Overflow Posts. In *17th International Conference on Mining Software Repositories, October 5–6, 2020, Seoul, Republic of Korea. New York, NY, USA*. ACM.

[6] Syed Ahmed and Mehdi Bagherzadeh. 2018. What Do Concurrency Developers Ask about? A Large-Scale Study Using Stack Overflow. In *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement* (Oulu, Finland) *(ESEM '18)*. Association for Computing Machinery, New York, NY, USA, Article 30, 10 pages. https://doi.org/10.1145/3239235.3239524

[7] Toufique Ahmed, Premkumar Devanbu, Christoph Treude, and Michael Pradel. 2024. Can LLMs replace manual annotation of software engineering artifacts? *arXiv preprint arXiv:2408.05534* (2024).

[8] Emran Altamimi, Abdullah Elkawakjy, and Cagatay Catal. 2023. Metamorphic rela-

tion automation: Rationale, challenges, and solution directions. *Journal of Software: Evolution and Process* 35, 1 (2023), e2509.

[9] Hadi Ardiny and Esmaeel Khanmirza. 2018. The Role of AR and VR Technologies in Education Developments: Opportunities and Challenges. In *2018 6th RSI International Conference on Robotics and Mechatronics (IcRoM)*. 482–487. https://doi.org/10.1109/ICRoM.2018.8657615

[10] Narges Ashtari, Andrea Bunt, Joanna McGrenere, Michael Nebeling, and Parmit K. Chilana. 2020. Creating Augmented and Virtual Reality Applications: Current Practices, Challenges, and Opportunities. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) *(CHI '20)*. Association for Computing Machinery, New York, NY, USA, 1–13. https://doi.org/10.1145/3313831.3376722

[11] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).

[12] Jinsheng Ba, Yuancheng Jiang, and Manuel Rigger. 2025. Metamorphic Coverage. *arXiv preprint arXiv:2508.16307* (2025).

[13] Sebastian Baltes, Florian Angermeir, Chetan Arora, Marvin Muñoz Barón, Chunyang Chen, Lukas Böhme, Fabio Calefato, Neil Ernst, Davide Falessi, Brian Fitzgerald, et al. 2025. Guidelines for empirical studies in software engineering involving large language models. *arXiv preprint arXiv:2508.15503* (2025).

[14] Earl T Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. 2014. The oracle problem in software testing: A survey. *IEEE transactions on software engineering* 41, 5 (2014), 507–525.

[15] Dibyendu Brinto Bose. 2025. From prompts to properties: Rethinking llm code generation with property-based testing. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 1660–1665.

[16] Dibyendu Brinto Bose, Yoseph Berhanu Alebachew, and Chris Brown. 2025. LLMs in Debate: Does Arguing Make Them Better at Detecting Metamorphic Relations?. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*. 43–50. https://doi.org/10.1109/ASEW67777.2025.00019

[17] Dibyendu Brinto Bose and Chris Brown. 2024. An Empirical Study on Current Practices and Challenges of Core AR/VR Developers. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering Workshops* (Sacramento, CA, USA) *(ASEW '24)*. Association for Computing Machinery, New York, NY, USA, 233–238. https://doi.org/10.1145/3691621.3694956

[18] Dibyendu Brinto Bose, Brendan David-John, and Chris Brown. 2024. Optimizing ar application testing: Integrating metamorphic testing to address developer and end-user challenges. In *International Conference on Human-Computer Interaction*. Springer, 20–33.

[19] Arda Celik and Qusay H Mahmoud. 2025. A Review of Large Language Models for Automated Test Case Generation. *Machine Learning and Knowledge Extraction* 7, 3 (2025), 97.

[20] Wing Chan, Jeffrey Ho, and T.H. Tse. 2007. Piping Classification to Metamorphic Testing: An Empirical Study towards Better Effectiveness for the Identification of Failures in Mesh Simplification Programs. *Proceedings - International Computer Software and Applications Conference* 1, 397 – 404. https://doi.org/10.1109/COMPSAC.2007.167

[21] Salma Charkaoui, Zakaria Adraoui, and El Habib Benlahmar. 2014. Cross-platform mobile development approaches. In *2014 Third IEEE International Colloquium in Information Science and Technology (CIST)*. 188–191. https://doi.org/10.1109/CIST.2014.7016616

[22] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[23] T Chen, D Huang, T.H. Tse, and Zhi Quan Zhou. 2004. Case Studies on the Selection of Useful Relations in Metamorphic Testing.

[24] Tsong Y Chen, Shing C Cheung, and Shiu Ming Yiu. 2020. Metamorphic testing: a new approach for generating next test cases. *arXiv preprint arXiv:2002.12543* (2020).

[25] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. 2018. Metamorphic Testing: A Review of Challenges and Opportunities. *ACM Comput. Surv.* 51, 1, Article 4 (jan 2018), 27 pages. https://doi.org/10.1145/3143561

[26] Stephanie Chuah. 2018. Why and who will adopt extended reality technology? Literature review, synthesis, and future research agenda. (12 2018).

[27] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. 2024. Scaling instruction-finetuned language models. *Journal of Machine Learning Research* 25, 70 (2024), 1–53.

[28] Jacob Cohen. 1960. A Coefficient of Agreement for Nominal Scales. *Educational and Psychological Measurement* 20, 1 (1960), 37–46. https://doi.org/10.1177/001316446002000104 arXiv:https://doi.org/10.1177/001316446002000104

[29] Tristan Coignion, Clément Quinton, and Romain Rouvoy. 2024. A performance study of llm-generated code on leetcode. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*. 79–89.

[30] Alinne C Correa Souza, Fátima LS Nunes, and Márcio E Delamaro. 2018. An automated functional testing approach for virtual reality applications. *Software Testing, Verification and Reliability* 28, 8 (2018), e1690.

[31] William L Crabtree, Benjamin F. Miller. 1992. Doing qualitative research. *Calif : Sage Publications* (1992).

[32] Stevão Alves de Andrade, Fatima LS Nunes, and Márcio Eduardo Delamaro. 2023. Exploiting deep reinforcement learning and metamorphic testing to automatically test virtual reality applications. *Software Testing, Verification and Reliability* 33, 8 (2023), e1863.

[33] Kai-Uwe Doer, Jens Schiefel, and W. Kubbat. 2001. Virtual Cockpit Simulation for Pilot Training. (03 2001), 8.

[34] Helen Dong, Shurui Zhou, Jin LC Guo, and Christian Kästner. 2021. Splitting, renaming, removing: a study of common cleaning activities in jupyter notebooks. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*. IEEE, 114–119.

[35] Alinne C Correa dos Santos, Marcio Eduardo Delamaro, and Fatima LS Nunes. 2013. The relationship between requirements engineering and virtual reality systems: A

systematic literature review. In *2013 XV Symposium on Virtual and Augmented Reality*. IEEE, 53–62.

[36] Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. 2023. Improving factuality and reasoning in language models through multiagent debate. In *Forty-first International Conference on Machine Learning*.

[37] Alejandra Duque-Torres. 2024. Selecting and Constraining Metamorphic Relations. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*. 212–216.

[38] M Eswaran and M V A Raju Bahubalendruni. 2022. Challenges and opportunities on AR/VR technologies for manufacturing systems in the context of industry 4.0: A state of the art review. *Journal of Manufacturing Systems* 65 (2022), 260–278. https://doi.org/10.1016/j.jmsy.2022.09.016

[39] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M Zhang. 2023. Large language models for software engineering: Survey and open problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. IEEE, 31–53.

[40] Yu-Min Fang and Chun Lin. 2019. The Usability Testing of VR Interface for Tourism Apps. *Applied Sciences* 9 (08 2019), 3215. https://doi.org/10.3390/app9163215

[41] Christopher Foster, Abhishek Gulati, Mark Harman, Inna Harper, Ke Mao, Jillian Ritchey, Hervé Robert, and Shubho Sengupta. 2025. Mutation-Guided LLM-based Test Generation at Meta. *arXiv preprint arXiv:2501.12862* (2025).

[42] Maribeth Gandy and Blair MacIntyre. 2014. Designer's Augmented Reality Toolkit, Ten Years Later: Implications for New Media Authoring Tools. In *Proceedings of the*

27th Annual ACM Symposium on User Interface Software and Technology (Honolulu, Hawaii, USA) *(UIST '14)*. Association for Computing Machinery, New York, NY, USA, 627–636. https://doi.org/10.1145/2642918.2647369

[43] Harrison Goldstein, Joseph W. Cutler, Daniel Dickstein, Benjamin C. Pierce, and Andrew Head. 2024. Property-Based Testing in Practice. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 187, 13 pages. https://doi.org/10.1145/3597503.3639581

[44] Qiuhan Gu. 2023. LLM-Based Code Generation Method for Golang Compiler Testing. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (San Francisco, CA, USA) *(ESEC/FSE 2023)*. Association for Computing Machinery, New York, NY, USA, 2201–2203. https://doi.org/10.1145/3611643.3617850

[45] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V Chawla, Olaf Wiest, and Xiangliang Zhang. 2024. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680* (2024).

[46] Lai Yip Hang, Rohaida Romli, and Nadia Diyana Mohd Muhaiyuddin. 2023. AR-OOP: Augmented Reality-based Mobile Learning Application for Learning Object-Oriented Programming. *Multidisciplinary Applied Research and Innovation* 4, 1 (2023), 116–121.

[47] Anh Nguyen Hoang, Minh Le-Anh, Bach Le, and Nghi DQ Bui. 2025. CodeWiki: Evaluating AI's Ability to Generate Holistic Documentation for Large-Scale Codebases. *arXiv preprint arXiv:2510.24428* (2025).

[48] Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, et al. 2023. Metagpt: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352* 3, 4 (2023), 6.

[49] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024).

[50] Haley Hunter-Zinck, Alexandre Fioravante De Siqueira, Váleri N Vásquez, Richard Barnes, and Ciera C Martinez. 2021. Ten simple rules on writing clean and reliable open-source scientific software. , e1009481 pages.

[51] Sangwon Hyun, Mingyu Guo, and M Ali Babar. 2024. Metal: Metamorphic testing framework for analyzing large-language model qualities. In *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 117–128.

[52] Bryce Ikeda and Daniel Szafir. 2021. An AR debugging tool for robotics programmers. In *4th International Workshop on Virtual, Augmented, and Mixed Reality for HRI*.

[53] Md Ashraful Islam, Mohammed Eunus Ali, and Md Rizwan Parvez. 2025. Codesim: Multi-agent code generation and problem solving through simulation-driven planning and debugging. In *Findings of the Association for Computational Linguistics: NAACL 2025*. 5113–5139.

[54] Tahir Jameel, Mengxiang Lin, and Liu Chao. 2015. Test oracles based on metamorphic relations for image processing applications. In *2015 IEEE/ACIS 16th International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)*. 1–6. https://doi.org/10.1109/SNPD.2015.7176238

[55] Lingxiao Jiang, Ghassan Misherghi, Zhendong Su, and Stephane Glondu. 2007. Deckard: Scalable and accurate tree-based detection of code clones. In *29th International Conference on Software Engineering (ICSE'07)*. IEEE, 96–105.

[56] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770* (2023).

[57] Dorota Kamińska, Tomasz Sapinski, Slawomir Wiak, Toomas Tikk, Rain Eric Haamer, Egils Avots, Ahmed Mohamed Helmi, Cagri Ozcinar, and Gholamreza Anbarjafari. 2019. Virtual Reality and Its Applications in Education: Survey. *Inf.* 10 (2019), 318.

[58] Upulee Kanewala and James M. Bieman. 2013. Using machine learning techniques to detect metamorphic relations for programs without test oracles. In *2013 IEEE 24th International Symposium on Software Reliability Engineering (ISSRE)*. 1–10. https://doi.org/10.1109/ISSRE.2013.6698899

[59] Hong Jin Kang, Khai Loong Aw, and David Lo. 2022. Detecting false alarms from automatic static analysis tools: How far are we?. In *Proceedings of the 44th International Conference on Software Engineering*. 698–709.

[60] Gagandeep Kaur and Sengupta Jyotsna. 2011. An Efficient Metamorphic Testing Technique Using Genetic Algorithm. *Communications in Computer and Information Science* 141, 180–188. https://doi.org/10.1007/978-3-642-19423-8_19

[61] Aisha Khatun and Daniel G Brown. 2024. Trutheval: A dataset to evaluate llm truthfulness and reliability. *arXiv preprint arXiv:2406.01855* (2024).

[62] Veronika Krauß, Alexander Boden, Leif Oppermann, and René Reiners. 2021. Current Practices, Challenges, and Design Implications for Collaborative AR/VR Application

Development. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems* (Yokohama, Japan) *(CHI '21)*. Association for Computing Machinery, New York, NY, USA, Article 454, 15 pages. https://doi.org/10.1145/3411764.3445335

[63] Craig A. Kuechenmeister, Patrick H. Linton, Thelma V. Mueller, and Hilton B. White. 1977. Eye Tracking in Relation to Age, Sex, and Illness. *Archives of General Psychiatry* 34, 5 (05 1977), 578–579. https://doi.org/10.1001/archpsyc.1977.01770170088008 arXiv:https://jamanetwork.com/journals/jamapsychiatry/articlepdf/491720/archpsyc_34_5_008.p

[64] JR Landis and GG Koch. 1977. The measurement of observer agreement for categorical data. *Biometrics* 33, 1 (March 1977), 159—174. https://doi.org/10.2307/2529310

[65] Woon-Sung Lee, Jung-Ha Kim, and Jun-Hee Cho. 1998. A driving simulator as a virtual reality tool. In *Proceedings. 1998 IEEE International Conference on Robotics and Automation (Cat. No.98CH36146)*, Vol. 1. 71–76 vol.1. https://doi.org/10.1109/ROBOT.1998.676264

[66] Sarah M. Lehman, Haibin Ling, and Chiu C. Tan. 2020. ARCHIE: A User-Focused Framework for Testing Augmented Reality Applications in the Wild. In *2020 IEEE Conference on Virtual Reality and 3D User Interfaces (VR)*. 903–912. https://doi.org/10.1109/VR46266.2020.00013

[67] Benjamin J Li, Jorge Peña, and Younbo Jung. 2023. VR/AR and wellbeing: The use of immersive technologies in promoting health outcomes. , 1119919 pages.

[68] Haiyang Li. 2025. Mrg-bench: Evaluating and exploring the requirements of context for repository-level code generation. *arXiv preprint arXiv:2508.02998* (2025).

[69] Han Li, Yuling Shi, Shaoxin Lin, Xiaodong Gu, Heng Lian, Xin Wang, Yantao Jia, Tao Huang, and Qianxiang Wang. 2025. Swe-debate: Competitive multi-agent debate for software issue resolution. *arXiv preprint arXiv:2507.23348* (2025).

[70] Nan Li and Jeff Offutt. 2016. Test oracle strategies for model-based testing. *IEEE Transactions on Software Engineering* 43, 4 (2016), 372–395.

[71] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Mishig Davaadorj, Joel Lamy-Poirier, João Monteiro, Oleh Shliazhko, Nicolas Gontier, Nicholas Meade, Armel Zebaze, Ming-Ho Yee, Logesh Kumar Umapathi, Jian Zhu, Benjamin Lipkin, Muhtasham Oblokulov, Zhiruo Wang, Rudra Murthy, Jason Stillerman, Siva Sankalp Patel, Dmitry Abulkhanov, Marco Zocca, Manan Dey, Zhihan Zhang, Nour Fahmy, Urvashi Bhattacharyya, Wenhao Yu, Swayam Singh, Sasha Luccioni, Paulo Villegas, Maxim Kunakov, Fedor Zhdanov, Manuel Romero, Tony Lee, Nadav Timor, Jennifer Ding, Claire Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank Mishra, Alex Gu, Jennifer Robinson, Carolyn Jane Anderson, Brendan Dolan-Gavitt, Danish Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jernite, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. 2023. StarCoder: may the source be with you! arXiv:2305.06161 [cs.CL] https://arxiv.org/abs/2305.06161

[72] Rui Li, Huai Liu, Pak-Lok Poon, Dave Towey, Chang-Ai Sun, Zheng Zheng, Zhi Quan Zhou, and Tsong Yueh Chen. 2024. Metamorphic relation generation: State of the art and visions for future research. *arXiv preprint arXiv:2406.05397* (2024).

[73] Rui Li, Huai Liu, Pak-Lok Poon, Dave Towey, Chang-Ai Sun, Zheng Zheng, Zhi Quan

Zhou, and Tsong Yueh Chen. 2025. Metamorphic relation generation: State of the art and research directions. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–25.

[74] Ziyu Li and Donghwan Shin. 2024. Mutation-based Consistency Testing for Evaluating the Code Understanding Capability of LLMs. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering - Software Engineering for AI* (Lisbon, Portugal) *(CAIN '24)*. Association for Computing Machinery, New York, NY, USA, 150–159. https://doi.org/10.1145/3644815.3644946

[75] Fang Liu, Yang Liu, Lin Shi, Houkun Huang, Ruifeng Wang, Zhen Yang, Li Zhang, Zhongqi Li, and Yuchi Ma. 2024. Exploring and evaluating hallucinations in llm-powered code generation. *arXiv preprint arXiv:2404.00971* (2024).

[76] Huai Liu, Fei-Ching Kuo, Dave Towey, and Tsong Yueh Chen. 2013. How effectively does metamorphic testing alleviate the oracle problem? *IEEE Transactions on Software Engineering* 40, 1 (2013), 4–22.

[77] Kaibo Liu, Yiyang Liu, Zhenpeng Chen, Jie M Zhang, Yudong Han, Yun Ma, Ge Li, and Gang Huang. 2024. Llm-powered test case generation for detecting tricky bugs. *arXiv preprint arXiv:2404.10304* (2024).

[78] Shang Liu, Wenji Fang, Yao Lu, Jing Wang, Qijun Zhang, Hongce Zhang, and Zhiyao Xie. 2024. RTLCoder: Fully Open-Source and Efficient LLM-Assisted RTL Code Generation Technique. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2024), 1–1. https://doi.org/10.1109/TCAD.2024.3483089

[79] Xiangyan Liu, Bo Lan, Zhiyuan Hu, Yang Liu, Zhicheng Zhang, Fei Wang, Michael Qizhe Shieh, and Wenmeng Zhou. 2025. Codexgraph: Bridging large language

models and code repositories via code graph databases. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 142–160.

[80] Yanwen Liu, Ruifeng Li, Hao Tao, and Zheng Zheng. 2024. Test Adequacy Criteria for Metamorphic Testing. In *2024 IEEE 24th International Conference on Software Quality, Reliability, and Security Companion (QRS-C)*. IEEE, 527–534.

[81] Junyi Lu, Lei Yu, Xiaojia Li, Li Yang, and Chun Zuo. 2023. Llama-reviewer: Advancing code review automation with large language models through parameter-efficient fine-tuning. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 647–658.

[82] Sanwal Manish. 2024. An autonomous multi-agent llm framework for agile software development. *International Journal of Trend in Scientific Research and Development* 8, 5 (2024), 892–898.

[83] Noble Saji Mathews and Meiyappan Nagappan. 2024. Design choices made by LLM-based test generators prevent them from finding bugs. *arXiv preprint arXiv:2412.14137* (2024).

[84] Noble Saji Mathews and Meiyappan Nagappan. 2024. Test-Driven Development and LLM-based Code Generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1583–1594.

[85] Johannes Mayer and Ralph Guderlei. 2006. An Empirical Study on the Selection of Good Metamorphic Relations. In *30th Annual International Computer Software and Applications Conference (COMPSAC'06)*, Vol. 1. 475–484. https://doi.org/10.1109/COMPSAC.2006.24

[86] Gergana Mileva. 2021. How Is Augmented Reality Transforming Special Education? *AR Post* (2021). https://arpost.co/2021/06/25/augmented-reality-special-education/.

[87] Geri Mileva. 2022. The Ultimate Guide to Virtual Reality Marketing in 2023. *Influencer Marketing Hub* (2022). https://influencermarketinghub.com/virtual-reality-marketing/.

[88] Christian Murphy, Gail Kaiser, Lifeng Hu, and Leon Wu. 2008. Properties of Machine Learning Applications for Use in Metamorphic Testing. 867–872.

[89] Mashaal Musleh, Mourad Ouzzani, Nan Tang, and AnHai Doan. 2020. Coclean: Collaborative data cleaning. In *Proceedings of the 2020 ACM SIGMOD international conference on management of data*. 2757–2760.

[90] Daren Nicholson, Colin Chalk, W Funnell, and Sam Daniel. 2006. Can virtual reality improve anatomy education? A randomised controlled study of a computer-generated three-dimensional anatomical ear model. *Medical education* 40 (12 2006), 1081–7. https://doi.org/10.1111/j.1365-2929.2006.02611.x

[91] Yaroslav Oliinyk, Michael Scott, Ryan Tsang, Chongzhou Fang, Houman Homayoun, et al. 2024. Fuzzing {BusyBox}: Leveraging {LLM} and Crash Reuse for Embedded Bug Unearthing. In *33rd USENIX Security Symposium (USENIX Security 24)*. 883–900.

[92] Amirkia Rafiei Oskooei, Selcan Yukcu, Mehmet Cevheri Bozoglan, and Mehmet S Aktas. 2025. Repository-level code understanding by llms via hierarchical summarization: Improving code search and bug localization. In *International Conference on Computational Science and Its Applications*. Springer, 88–105.

[93] Wendkûuni C Ouédraogo, Kader Kaboré, Haoye Tian, Yewei Song, Anil Koyuncu, Jacques Klein, David Lo, and Tegawendé F Bissyandé. 2024. Large-scale, Independent and Comprehensive study of the power of LLMs for test case generation. *arXiv preprint arXiv:2407.00225* (2024).

[94] Simon Parkin. 2015. How VR is training the perfect soldier. (2015). "https://www.wareable.com/vr/how-vr-is-training-the-perfect-soldier-1757/".

[95] Rui Prada, ISWB Prasetya, Fitsum Kifetew, Frank Dignum, Tanja EJ Vos, Jason Lander, Jean-yves Donnart, Alexandre Kazmierowski, Joseph Davidson, and Pedro M Fernandes. 2020. Agent-based testing of extended reality systems. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. IEEE, 414–417.

[96] Archiki Prasad, Peter Hase, Xiang Zhou, and Mohit Bansal. 2022. Grips: Gradient-free, edit-based instruction search for prompting large language models. *arXiv preprint arXiv:2203.07281* (2022).

[97] PK Ragunath, S Velmourougan, P Davachelvan, S Kayalvizhi, and R Ravimohan. 2010. Evolving a new model (SDLC Model-2010) for software development life cycle (SDLC). *International Journal of Computer Science and Network Security* 10, 1 (2010), 112–119.

[98] Nikitha Rao, Kush Jain, Uri Alon, Claire Le Goues, and Vincent J. Hellendoorn. 2023. CAT-LM Training Language Models on Aligned Code And Tests. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 409–420. https://doi.org/10.1109/ASE56229.2023.00193 ISSN: 2643-1572.

[99] Zeeshan Rasheed, Muhammad Waseem, Aakash Ahmad, Kai-Kristian Kemell, Wang Xiaofeng, Anh Nguyen Duc, and Pekka Abrahamsson. 2024. Can large language models

serve as data analysts? A multi-agent assisted approach for qualitative data analysis. *arXiv preprint arXiv:2402.01386* (2024).

[100] Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084* (2019).

[101] Kenneth A Ritter III, Christoph W Borst, and Terrence L Chambers. 2015. Overview and assessment of unity toolkits for rapid development of an educational vr application. *International Journal for Innovation Education and Research* 3, 7 (2015), 147–164.

[102] José Miguel Rojas, Mattia Vivanti, Andrea Arcuri, and Gordon Fraser. 2017. A detailed investigation of the effectiveness of whole test suite generation. *Empirical Software Engineering* 22, 2 (2017), 852–893.

[103] Dhia Elhaq Rzig, Nafees Iqbal, Isabella Attisano, Xue Qin, and Foyzul Hassan. 2023. Virtual reality (vr) automated testing in the wild: A case study on unity-based vr applications. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1269–1281.

[104] Kira Sam. 2024. Llama 3.1: An in-depth analysis of the next-generation large language model. *Available at SSRN 6139407* (2024).

[105] M Samadbeik, D Yaaghobi, P Bastani, S Abhari, R Rezaee, and A Garavand. 2018. The Applications of Virtual Reality Technology in Medical Groups Teaching. In *J Adv Med Educ Prof., October 5–6, 2018, New York, NY, USA*. Springer.

[106] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. 2023. Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design or: How I learned to start worrying about prompt formatting. *arXiv preprint arXiv:2310.11324* (2023).

[107] Sergio Segura, Gordon Fraser, Ana B. Sanchez, and Antonio Ruiz-Cortés. 2016. A Survey on Metamorphic Testing. *IEEE Transactions on Software Engineering* 42, 9 (2016), 805–824. https://doi.org/10.1109/TSE.2016.2532875

[108] Sergio Segura, Dave Towey, Zhi Quan Zhou, and Tsong Yueh Chen. 2018. Metamorphic testing: Testing the untestable. *IEEE Software* 37, 3 (2018), 46–53.

[109] Lijun Shan and Hong Zhu. 2009. Generating structurally complex test cases by data mutation: A case study of testing an automated modelling tool. *Comput. J.* 52, 5 (2009), 571–588.

[110] Shreya Shankar, JD Zamfirescu-Pereira, Björn Hartmann, Aditya Parameswaran, and Ian Arawjo. 2024. Who validates the validators? aligning llm-assisted evaluation of llm outputs with human preferences. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*. 1–14.

[111] Alinne C. Corrêa Souza, Fátima de Lourdes dos Santos Nunes Marques, and Márcio Eduardo Delamaro. 2018. An automated functional testing approach for virtual reality applications. *Software Testing, Verification and Reliability* (2018). https://doi.org/10.1002/stvr.1690

[112] Maximilian Speicher, Brian D. Hall, Ao Yu, Bowen Zhang, Haihua Zhang, Janet Nebeling, and Michael Nebeling. 2018. XD-AR: Challenges and Opportunities in Cross-Device Augmented Reality Application Development. *Proc. ACM Hum.-Comput. Interact.* 2, EICS, Article 7 (jun 2018), 24 pages. https://doi.org/10.1145/3229089

[113] Claudio Spiess, David Gros, Kunal Suresh Pai, Michael Pradel, Md Rafiqul Islam Rabin, Susmit Jha, Prem Devanbu, and Toufique Ahmed. 2024. Quality and Trust in LLM-generated Code. *arXiv e-prints* (2024), arXiv–2402.

[114] Chang-ai Sun, Hui Jin, SiYi Wu, An Fu, ZuoYi Wang, and Wing Kwong Chan. 2024. Identifying metamorphic relations: A data mutation directed approach. *Software: Practice and Experience* 54, 3 (2024), 394–418. https://doi.org/10.1002/spe.3280 arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/spe.3280

[115] Wei Tao, Yucheng Zhou, Yanlin Wang, Wenqiang Zhang, Hongyu Zhang, and Yu Cheng. 2024. Magis: Llm-based multi-agent framework for github issue resolution. *Advances in Neural Information Processing Systems* 37 (2024), 51963–51993.

[116] Christoph Treude. 2024. Qualitative data analysis in software engineering: Techniques and teaching insights. In *Handbook on Teaching Empirical Software Engineering*. Springer, 155–176.

[117] Rosalia Tufano, Simone Masiero, Antonio Mastropaolo, Luca Pascarella, Denys Poshyvanyk, and Gabriele Bavota. 2022. Using pre-trained models to boost code review automation. In *Proceedings of the 44th international conference on software engineering*. 2291–2302.

[118] Elena Vergou, Ioanna Pagouni, Marios Nanos, and Katia Lida Kermanidis. 2023. Readability classification with wikipedia data and all-minilm embeddings. In *IFIP International Conference on Artificial Intelligence Applications and Innovations*. Springer, 369–380.

[119] Peng Wu, SHI Xiao-Chun, TANG Jiang-Jun, and LIN Hui-Min. 2005. Metamorphic Testing and Special Case Testing: A Case Study. *Journal of Software* 16 (07 2005). https://doi.org/10.1360/jos161210

[120] Kai Xiong, Xiao Ding, Yixin Cao, Ting Liu, and Bing Qin. 2023. Examining inter-consistency of large language models collaboration: An in-depth analysis via debate. *arXiv preprint arXiv:2305.11595* (2023).

[121] Congying Xu, Songqiang Chen, Jiarong Wu, Shing-Chi Cheung, Valerio Terragni, Hengcheng Zhu, and Jialun Cao. 2024. Mr-adopt: Automatic deduction of input transformation function for metamorphic testing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 557–569.

[122] Haoran Yang, Hongyuan Lu, and Wai Lam. 2024. Rephrasing Invokes Better Generations for Large Language Models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 4: Student Research Workshop)*. 8–15.

[123] Jian Yang, Jiajun Zhang, Jiaxi Yang, Ke Jin, Lei Zhang, Qiyao Peng, Ken Deng, Yibo Miao, Tianyu Liu, Zeyu Cui, et al. 2024. Execrepobench: Multi-level executable code completion evaluation. *arXiv preprint arXiv:2412.11990* (2024).

[124] Shin Yoo. 2010. Metamorphic testing of stochastic optimisation. In *2010 Third International Conference on Software Testing, Verification, and Validation Workshops*. IEEE, 192–201.

[125] Chunyan Zhang, Junchao Wang, Qinglei Zhou, Ting Xu, Ke Tang, Hairen Gui, and Fudong Liu. 2022. A survey of automatic source code summarization. *Symmetry* 14, 3 (2022), 471.

[126] Jie Zhang, Junjie Chen, Dan Hao, Yingfei Xiong, Bing Xie, Lu Zhang, and Hong Mei. 2014. Search-based inference of polynomial metamorphic relations. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering* (Vasteras, Sweden) *(ASE '14)*. Association for Computing Machinery, New York, NY, USA, 701–712. https://doi.org/10.1145/2642937.2642994

[127] Danna Zheng, Danyang Liu, Mirella Lapata, and Jeff Z Pan. 2024. TrustScore:

reference-free evaluation of LLM response trustworthiness. *arXiv preprint arXiv:2402.12545* (2024).

[128] Zhi Quan Zhou, Shaowen Xiang, and Tsong Yueh Chen. 2015. Metamorphic testing for software quality assessment: A study of search engines. *IEEE Transactions on Software Engineering* 42, 3 (2015), 264–284.

[129] Yuxiang Zhu and Minxue Pan. 2019. Automatic code summarization: A systematic literature review. *arXiv preprint arXiv:1909.04352* (2019).

[130] Zhengyang Zhu, Hong-Ning Dai, Hanyang Guo, Zeqin Liao, and Zibin Zheng. 2025. VRExplorer: A Model-based Approach for Semi-Automated Testing of Virtual Reality Scenes. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 482–494.